SIM4000 User Manual

The native emulator and its supplied RC system

SIM4000 project

About this manual

This manual explains how to build and use the native emulator. You do not need to know its C implementation, historical Pascal source, or CPU internals to run the supplied RC4000 system.

Development and testing use Linux on x86-64. The runtime uses C17, POSIX interfaces and GNU Readline. Other hosts, including macOS, have not been validated; platform-specific setup and testing are left to interested contributors.

For supported-system boundaries and known differences from hardware and the DOS emulator, see *Compatibility and Limitations* (`doc/COMPATIBILITY.md`).

The local terminal can address three command interpreters: the simulator, the RC system process `s`, and FP inside a user process. Commands for one must not be entered at another.

For the separate automatic TEM/PRIMO/SOS environment, read MIPS/TS: a terminal session (p. 53). It has its own disk pair, demonstration login and resource limits. The default-profile process-creation and host-import exercises are not additional steps in a MIPS/TS login.

Reading commands and responses

Examples throughout the manual have context titles: host shell, simulator prompt, guest system process `s`, guest FP, a tool's own input, or configuration/file contents. These titles are not text to enter. Output examples are labelled separately; do not type the monitor's messages or prompts. Return submits each command line unless the accompanying instructions say otherwise.

Angle-bracketed key names such as `<Control-E>` describe a key press, not literal input. Likewise, `local:` and `remote:` in two-terminal transcripts identify the terminal; omit these prefixes when typing. Ellipses in displayed results stand for session-dependent fields, not text to enter. Monospaced names in prose identify commands, files or configuration settings; their context still determines where to use them.

Local emulator-escape examples use Control-E, the shipped default. Substitute your configured `OPERATOR ESCAPE` key if you change it; Control-E at Readline's operator prompt is instead a line-editing binding. Remote Telnet attention uses its endpoint's own setting, not the local emulator escape key. See keyboard settings (p. 94) and remote attention (p. 98).

Syntax-reference blocks are not scripts: brackets indicate optional arguments, vertical bars indicate alternatives, and uppercase placeholders name values you must supply. Fresh-session examples are independent unless their text explicitly continues an earlier procedure. A command returning without an error is evidence for that example, not every function of the historical tool.

Cross-references within this manual include a page number for use with a printed manual. References to `doc/` files are repository-relative paths, not runtime files or files inside the guest disk image.

Terms used in this manual

Simulator The emulator program running on the host. Its `sim4000>` prompt controls execution, media and debugging; it is outside the guest operating system.

System process s The guest supervisor's command interface for creating and controlling processes.

FP The file processor: a guest command interpreter inside an ordinary user process. It starts utilities, compilers and other programs.

Catalog entry and scope A guest name and its metadata. Temporary scope belongs to a process's work; retained user scope can survive process removal. Neither scope by itself determines whether host disk writes survive exit.

Segment and RC byte Guest allocation and addressing units. A backing-store segment holds 256 24-bit words (768 host bytes). An RC byte is a 12-bit halfword, not an eight-bit host byte.

Overlay Changes kept separately from the base disk image. A volatile overlay disappears on emulator exit; a persistent overlay is saved in its own host file and must stay with the matching base image.

Attention and suspension Guest attention selects which process receives terminal input. Emulator suspension stops CPU execution and opens the simulator prompt. They are not interchangeable.

The original DOS emulator was written by Henrik Jacobsen. This manual describes the native reimplementation and the supplied historical software, not the original hardware in every detail.

Read Build and start (p. 2) followed by First monitor session (p. 3) for the shortest path to a running system. Copying the runtime files is an optional alternative, not an extra startup step. Part II supplies task-oriented guest procedures; Part III is the emulator and device reference. Examples distinguish host shell commands, simulator commands and guest commands; always check the current command context.

Contents

About this manual ii

I. Getting started 1

1. Build and start 2

2. First monitor session 3
2.1. Startup and terminal recovery . . . 4

3. Run from your own working directory 5

4. Everyday operation 7
4.1. Find the right procedure . . . 7
4.2. Decide what to keep before starting . . . 7
4.3. Suspend, finish or exit . . . 8
4.4. When the terminal context is unclear . . . 8

II. Working with the guest system 9

5. Processes and command contexts 10
5.1. Create and inspect a process . . . 10
5.2. Inspect resources with guest `claim` . . . 11
5.3. Stop, resume, and remove a process . . . 12
5.4. Set the monitor date later . . . 12
5.5. Troubleshooting terminal routing . . . 13

6. Files, catalog entries, and Algol programs 15
6.1. Import, copy, rename, and export . . . 15
6.2. Display a text file on the guest console . . . 16
6.3. Compile and run a small Algol program . . . 16
6.4. Clean up and keep results . . . 17

7. Keep guest files across sessions 18
7.1. Prepare a writable working copy . . . 18
7.2. Allocate resources and promote a file . . . 19
7.3. End the process and read the retained file . . . 19
7.4. Reclaim the example file deliberately . . . 20
7.5. Inspect catalog information . . . 20

8. Software and files in the supplied disk image 28
8.1. Everyday commands and development tools . . . 28
8.2. Edit a text file inside the guest . . . 30
8.3. Compare text files inside the guest . . . 32
8.4. Assemble a SLANG source . . . 34
8.5. Save an assembler listing and use an FP command file . . . 35
8.6. Call one command file from another . . . 36
8.7. Execute commands conditionally . . . 38
8.8. Recover from FP and assembly errors . . . 41
8.9. Demonstrations and diagnostic programs . . . 43
8.10. Startup, system files, and optional subsystems . . . 47
8.11. Libraries and catalog entry points . . . 48
8.12. Other installed tools: not yet an operating guide . . . 49

9. Host file exchange **50**
9.1. Choose the native disk image . . . 50
9.2. Configure the exchange directory . . . 50
9.3. Import and export text . . . 51
9.4. Choose text or binary transfer . . . 51
9.5. Protect host files and interpret failures . . . 52

10. MIPS/TS: a terminal session **53**
10.1. What this environment provides . . . 53
10.2. Select the files and start . . . 54
10.3. Connect and log in . . . 54
10.4. Small jobs and resource limits . . . 56
10.5. Files, persistence and shutdown . . . 59
10.6. Check status without disturbing the job . . . 63
10.7. Print through PRIMO . . . 64
10.8. Troubleshooting and current limits . . . 66
10.9. Historical reference and coverage . . . 68

III. Emulator and device reference 69

11. Configuration and disk persistence **70**
11.1. Select CPU and device capabilities . . . 70
11.2. Choose a maintained-monitor profile . . . 70
11.3. Add a second RC433X disk . . . 71
11.4. Select the optional historical disk interface . . . 72
11.5. Select runtime files and override paths . . . 72
11.6. Choose how RC433X disk writes survive . . . 73
11.7. Use an explicit complete working copy . . . 73
11.8. Inspect stored catalog records and runtime limits . . . 74

12. Starting and controlling the emulator **75**
12.1. Select a startup mode . . . 75
12.2. Override configuration and bound execution . . . 75
12.3. Inspect files without booting the guest . . . 76
12.4. Edit commands at the operator prompt . . . 77
12.5. Simulator commands and aliases . . . 77
12.6. Modify stopped machine state . . . 78
12.7. Create a blank disk image . . . 79
12.8. Inspect and change disk attachments . . . 79

13. Paper-tape files **81**
13.1. Mount and inspect a reader tape . . . 81
13.2. Distinguish paper-tape autoload from disc startup . . . 81
13.3. Create and close punch output . . . 82

14. RC610 line printer **83**
14.1. Automatic output and file selection . . . 83
14.2. A first print from s . . . 84
14.3. Print an installed Algol source . . . 84
14.4. Text mapping, visibility and limits . . . 85

15. Optional V2 terminal protocol **86**
15.1. Keep classic operation as the normal profile . . . 86
15.2. Understand input tables and frontend policies . . . 86
15.3. Observe the experimental compatibility boundary . . . 87

16. Magnetic-tape files (RC749) **88**
16.1. Enable and mount an RC749 device . . . 88
16.2. Recognize the supported tape format . . . 88
16.3. Opt in to guest-requested tape mounting . . . 89

17. Debugging and core files **90**
17.1. Stop before execution or after a core write . . . 90
17.2. Inspect core without modifying it . . . 90

17.3. Exchange packed core data with host . . . 91
17.4. Execute a bounded diagnostic step . . . 91

18. Execution throttle **92**
18.1. Choose an interactive pacing mode . . . 92
18.2. Keep a startup setting in the configuration . . . 92
18.3. Understand what pacing does not control . . . 93

19. Keyboard and date settings **94**
19.1. Set local shortcuts and initialize the date . . . 94
19.2. Map Backspace and decode terminal input . . . 94
19.3. Inspect or discard pasted input . . . 95
19.4. Keep operator input separate from bounded-run data . . . 96

20. Telnet consoles **97**
20.1. Enable a listener and connect a client . . . 97
20.2. Request guest attention, not emulator suspension . . . 98
20.3. Select a process on a second terminal . . . 98
20.4. Use the RC851 screen translation . . . 99

Part I.

Getting started

1. Build and start

Requirements: a POSIX host, a C17 compiler, CMake 3.20 or newer, GNU Readline development headers/library, and a terminal. The linked Readline runtime library must also be available when running a copied executable. Neither Python nor TeX is required for an ordinary build or binary-distribution execution. Optional developer verification and media regeneration have separate dependencies.

From the repository root:

Host shell: build and start

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build
cd build
cp -R ../doc/usermanual/examples .
./sim4000 -a
```

CMake supplies `RC4000.cfg`, `RC4000.dfc403.cfg`, `MONITOR`, `UNIT0`, and `UNIT0.original` in the build directory and creates `exchange`. The default configuration selects the native `UNIT0` with `fromhost` and `tohost`. The unchanged historical image retains `fromdos` and `todos`; select it with `./sim4000 -a --disk UNIT0.original`.

The command `-a` starts the interactive interface, requests disk startup, and immediately runs. Normal disk writes are volatile: they are visible during this emulator session but disappear when it exits. Choose a persistence mode before starting if you want to keep your work.

The copied `examples/` directory is optional for normal operation, but contains the input files used in the guest tutorials. It is not executed automatically. Keep it beside the runtime files if you move your working directory; the examples then need no source checkout.

You are now in the running emulator, not at the host shell. Continue directly with First monitor session (p. 3) to answer the date question. Skip the optional working-directory setup (p. 5) unless you want to run a separate copy.

2. First monitor session

The preceding build-and-start procedure has already started the emulator. If you are using a copied working directory instead, run `./sim4000 -a` there once. The following steps are guest input, not host shell commands.

The supplied monitor normally prints messages including:

Guest output: startup messages

```
from s
disc         mounted on 015
ready  **date not initialized

from startproc

initialize date:
```

Enter six two-digit fields, followed by Return:

Guest input: answer the date question

```
86 09 30 08 00 00
```

The fields are year, month, day, hour, minute, and second. The monitor accepts years 1982 through 1999. Alt-D can insert the translated host date; the default `DATE YEAROFFSET=-40` maps 2026 to 1986. If your terminal intercepts Alt-D, press Control-E, enter `send-date` at `sim4000>`, and enter `continue`. A manually entered date avoids dependency on host-date settings.

After `pause startproc finis fp`, press bare Escape. At `att`, type `s` and press Return to select the system process. You can now enter a system command such as `new`. If Escape is inconvenient, use the simulator commands instead:

Key press, then commands at the simulator prompt

```
<Control-E>
attention
continue
```

Then type `s` and Return in the RC terminal, not at `sim4000>`.

Create an ordinary user process before running programs or transferring files:

Guest system process s: create a user FP

```
new work size 100000 run
```

Wait for `to work`, then enter the program name in FP:

Guest FP in work: start the demonstration

```
geo
```

The example program asks `dansk/English ?`; enter `e` to select English. This demonstrates that the disk-resident program is running. Only enter `geo` when you actually want that program; the `new` command alone leaves the new process ready for other FP commands.

To suspend execution, press Control-E. Use `continue` (or `c`) to return to the RC session without resetting it. Use `quit` (or `q`) at the simulator prompt to exit. Quitting does not preserve a volatile disk overlay.

2.1. Startup and terminal recovery

Run from the build or working directory so the default configuration is found. Configured paths are relative to the configuration file; command-line overrides and runtime attachment paths are relative to the current directory.

If startup reports a missing `HOSTFILES` root, create the configured `exchange` directory. It must already exist when the emulator starts.

If a date is rejected, check the year range and configured year offset. Use a known valid historical date manually when necessary. If the terminal is left in an unusable state after a host crash or forced termination, run `stty sane` in the host shell.

Telnet attention selects an RC process; it does not open the simulator prompt. A process already attached to another terminal must first be released there, as explained in Telnet consoles (p. 97).

For the file tutorials, start a fresh disposable session and follow Create and inspect a process (p. 10). Do not enter their FP commands while `geo` is waiting for an answer. If you only tried the demonstration, Control-E followed by `quit` ends this disposable session; start `./sim4000 -a` again in the host shell and initialize its date before the next tutorial.

3. Run from your own working directory

This is an alternative to running from `build/`, not a step to perform inside an already running emulator. If you followed the first-session guide, export any wanted results and quit at `sim4000>` before starting this separate session. Enter the commands below in a host shell at the repository root.

No installation is necessary. For the default compatibility profile, copy these four files to a new working directory after building; the source tree, CMake, compiler, Python, and generation tools are not needed to run it:

File	Purpose
`sim4000`	Executable; retain its executable permission.
`RC4000.cfg`	Default configuration, found in the current directory.
`MONITOR`	Monitor image selected by the configuration.
`UNIT0`	Base system disk selected by the configuration.

The executable must match the destination host's operating system and architecture. The host must provide a compatible dynamic loader and shared libraries, including GNU Readline. Copying the executable does not make it independent of those runtime libraries or portable across operating systems. No build dependencies are needed on that host.

For example, from the repository root after the build above:

Host shell: create a working directory

```
run_dir=$(mktemp -d ./sim4000-work.XXXXXX)
cp build/sim4000 build/RC4000.cfg build/MONITOR build/UNIT0 "$run_dir"/
mkdir "$run_dir/exchange"
cp -R doc/usermanual/examples "$run_dir"/
cd "$run_dir"
./sim4000 -a
```

Create `exchange/` for the supplied configuration's host-file exchange device. It holds host input files and newly exported files, not guest catalog files. If you do not want host-file exchange, comment out `DEFINE HOSTFILES` in your copy of the configuration; the directory is then unnecessary. The default configuration needs no mounted reader or punch files and enables no Telnet listener.

The optional `examples/` directory copied above contains the tutorial inputs. It is not required to boot the emulator. In later procedures, `examples/` and `exchange/` refer to directories beneath your working directory. Review FP command files before running them; they are guest commands, not host scripts.

After starting, follow First monitor session (p. 3). Do not start another emulator on the same writable image or overlay.

Other optional files depend on the configuration you choose:

- `RC4000.compat.cfg`: an explicit alias of the default configuration. For protection, copy `RC4000.protected.cfg`, `MONITOR.protected`, `UNIT0.protected` and `PROFILES.txt` from the build tree beside the executable. Do not substitute the default disk or monitor. See Choose a maintained-monitor profile (p. 70) for transfer names and software limits. Both profiles are already included in the binary archive. For the separate MIPS/TS environment, copy `UNIT0.mipsts`, `UNIT1.mipsts` and `RC4000.mipsts.cfg` alongside the compatibility `MONITOR`; see MIPS/TS (p. 53) for startup, login and its persistent overlays.
- `UNIT0.original`: the unchanged historical disk, only needed if selected.
- `RC4000.dfc403.cfg` and `STDMON`: the supplied optional historical controller setup. Copy both with `UNIT0`; no generation tool is needed.
- Paper-tape input/output files: only needed when mounted or configured.
- `UNIT0.overlay`: created when persistent overlays are enabled. Preserve it together with its matching base `UNIT0` and configuration when moving or backing up your work. Stop the emulator before copying this set.
- This manual: useful for reference, but not used by the executable.

To retain guest disk changes, replace the active disk definition in your copied `RC4000.cfg` with:

Configuration file: copied RC4000.cfg

```
DEFINE RC433X DEVNO=15 FILE=UNIT0 OVERLAY=UNIT0.overlay
```

Without this change (or `--disk-overlay`), disk writes disappear on exit. Guest temporary files can also be removed by the monitor: use `scope user` as described in Keep guest files across sessions (p. 18). The base disk is read-only in both overlay modes.

Run `./sim4000 -a` inside the working directory. From a different directory, select the configuration explicitly, for example:

Host shell: select a configuration

```
./work/sim4000 -a -c work/RC4000.cfg
```

Configured monitor, disk, overlay, and host-exchange paths remain relative to the configuration file. Command-line path overrides are instead relative to the current working directory. Renaming or moving files therefore requires adjusting the selected configuration or providing the appropriate overrides.

4. Everyday operation

Use this section as a route through the detailed, checked examples. Choose one fresh-session example at a time: do not create another process with a name already in use or enter FP commands while a demonstration program is waiting for an answer.

4.1. Find the right procedure

Task	Procedure
Start, enter the date and reach `s`	First monitor session (p. 3)
Create a user FP, stop or remove a process	Processes and command contexts (p. 10)
Import, display, copy, rename or export text	Files and Algol examples (p. 15)
Edit an imported text file	Guest editor (p. 30)
Compile Algol or assemble SLANG	Algol example (p. 16), SLANG example (p. 34)
Find programs or inspect file metadata	Catalog information (p. 20), installed software (p. 28)
Preserve guest files for another session	Keep guest files (p. 18)
Connect another terminal	Telnet consoles (p. 97)
Choose text or binary host transfer	Transfer modes (p. 51)
Run a reviewed FP command file	Command-file input (p. 36), conditional execution (p. 38)
Recover from accidental pasted input	Inspect or discard input (p. 95)
Select disk persistence or another controller	RC433X persistence (p. 73), historical disk profile (p. 72)
Limit host CPU use	Execution throttle (p. 92)
Inspect guest memory and execution	Debugging and core files (p. 90)

4.2. Decide what to keep before starting

The default disk mode is disposable. Exporting a result with `tohost` creates a host file that survives emulator exit, but a guest file written only to the volatile disk overlay does not survive. For ordinary experiments, use that default and export wanted results before quitting.

For work that must remain on the guest disk, make both decisions explicitly:

- Select a persistent overlay or a writable working copy before booting.
- Give the guest file a retained catalog scope using the checked resource allocation and promotion example (p. 19).

Persistent disk writes alone do not prevent the monitor from deleting a process's temporary files. Retained guest scope alone does not make a volatile disk overlay persistent. Follow the complete retention example rather than changing just one of these settings.

4.3. Suspend, finish or exit

These are different actions, not interchangeable shutdown commands:

Action	Where	What remains
Control-E, then `continue`	Simulator	Guest execution resumes without reset.
`finis`	Idle ordinary FP command input	Ends that FP; the system removes its process and temporary entries.
`quit`	`sim4000>`	Exits the emulator; volatile disk changes and running guest state are lost.

For a finished ordinary FP session, export wanted files first, enter `finis` and wait for the system's pause notice. If you need to verify removal, use a completed system `list`; the pause notice can precede cleanup. Then press Control-E and enter `quit` at the simulator prompt. This is not a rule for exiting an editor, an interactive diagnostic or another running program; use its own documented exit procedure to get back to FP first.

To take a break without ending the guest session, use Control-E and leave the emulator running at `sim4000>`, then `continue` later. The session is still only in that host process; quitting does not create a resumable snapshot. A saved core file is not a complete guest-session checkpoint.

4.4. When the terminal context is unclear

At `sim4000>`, enter only simulator commands. At guest `att`, select a process name. In `s`, use system commands such as `list` or `new`. In an idle user FP, use utility names and file commands. A program-specific question is another context again: answer that program, not the host shell or FP.

If an expected system reply is delayed by `to <process>`, do not send another system command into that process's pending read. See the bounded queued FP read example (p. 13); its empty-line procedure applies only to the checked idle-FP situation. For other failures, use startup recovery (p. 4) or the specific FP and assembly error examples (p. 41).

Part II.

Working with the guest system

5. Processes and command contexts

These examples apply to the supplied monitor and native disk image. Except for the default-profile-only shared-terminal procedure at the end, they have also been checked with the optional standard monitor and DFC403 disk configuration. Historical manuals describe other versions; their command syntax is not always interchangeable with this system.

Context	How to reach it	Examples
host shell	Before starting the simulator, or another host terminal	`mkdir`, `cp`
Simulator, `sim4000>`	Control-E, unless configured differently	`continue`, `quit`, `attention`
RC system process, `s`	Escape, then `s` and Return at `att`	`list`, `max`, `new`, `proc`, `date`
FP in a user process	Create it with `new ... run`, or select its name at `att`	`lookup`, `copy`, `rename`, `clear`, program names

Escape requests attention from the guest; Control-E suspends the machine and opens the simulator interface. They do different jobs. After each completed `s` command, use Escape and `s` again for the next system command. FP accepts successive command lines without this extra step. Return submits a line; an idle FP process does not necessarily show a conventional prompt.

5.1. Create and inspect a process

Start from a freshly booted session, after date initialization. At `att`, select `s`, then enter:

Guest system process s: list user processes

```
list
```

This lists ordinary user processes; initially there may be none. Select `s` again and enter `max` to see the currently available process resources. Select `s` once more and create the process used below:

Guest system process s: allocate and start muser

```
new muser size 100000 temp disc 200 10 run
```

`size 100000` reserves 100000 RC address units (12-bit units, not host bytes). `temp disc 200 10` requests a temporary disk claim of 200 segments and ten catalog entries. `run` starts FP and attaches the terminal to it; the monitor reports `to muser`. Use a fresh session for these examples so that the process and file names do not already exist. If allocation fails, check `list` and `max` before requesting a smaller process or releasing an unused one.

No storage protection in the default profile

The supplied compatibility profile leaves storage protection disabled. Process resource claims are not a security boundary. Optional experimental protection requires an explicitly selected, separately prepared monitor and software profile; enabling it is not part of this tutorial.

5.2. Inspect resources with guest `claim`

Inside an ordinary FP process, enter bare `claim` to inspect the guest's resource report. This is not a simulator command and does not create a file. In the checked `muser` session it prints:

Guest output: resource report from claim

```
area 5   buf 7   size 100000   first core 44842

disc: 3 segm/slice
  temp      1002 segm     334 slices      28 entr
  login      801 segm     267 slices      20 entr
  perm       801 segm     267 slices      20 entr
```

The report includes `area`/`buf` resource counters, the process's core size and starting address, and disk counters labelled `temp`, `login` and `perm`. The tested `size` and `first core` agree with the process entry in `s`'s `list`. Exact values depend on the session; they are not universal defaults.

The following allocation exercise is optional. Before it, put a short ASCII/LF sample in `exchange/input` from a second host terminal in the emulator's working directory; you can copy `examples/hello.alg` there. Skip this exercise if you want to go directly to the file tutorial. With that host file and an unused guest name `spacetmp`, enter:

Guest FP in muser: inspect allocation and release

```
claim
spacetmp=fromhost input
lookup spacetmp
claim
clear temp spacetmp
claim
```

The import's descriptor reports 36 allocated segments. The `temp` line decreases by 36 segments, 12 slices and one entry; `login` and `perm` stay unchanged. Clearing the file restores the original counters. Repeated bare `claim` calls without intervening allocation give unchanged reports. The example is tested with both supported monitor/controller configurations.

There is an important version-specific limitation: the displayed `temp` counters are **not** simply the literal `200 10` requested in `new muser`. The complete relationship between these reported counters, inherited claims and enforcement in this modified monitor has not been established. Do not treat this report as proof of an isolated, hard 200-segment quota, or infer that unused permanent counters authorize changing installed system files. Only bare reporting and the bounded temporary allocation above are verified; other `claim` arguments and resource-limit changes remain untested.

Use `max` in `s` for its process-resource report, `show` in FP for free disk allocation, and `cata` in FP for main-catalog slots. They are different views, not interchangeable ways to count remaining resources.

5.3. Stop, resume, and remove a process

This is an optional process-control exercise. If you want to continue directly with the file tutorial (p. 15), leave `muser` running and skip this section for now. Ending or removing it discards its execution state; export wanted files before proceeding.

With `muser` running, press Escape, select `s`, and enter:

Guest system process s: stop muser

```
proc muser stop
```

Wait for the system reply `from s` / `ready`; an earlier `to muser` message does not by itself mean the system operation has finished. Select `s` again and issue `list`. The row for `muser` now ends in `stop s`. Its columns are name, core address, size, protection key, stop count, state, and parent. Addresses and available resources vary with the monitor variant and session.

Select `s` again and resume FP:

Guest system process s: start muser

```
proc muser start
```

Wait for the completed system reply. Before entering any FP command, request attention and select `muser` at `att`; selecting `s` for process control does not make subsequent input an FP command.

To end this FP session normally, enter `finis` in the user process. The supplied system process reports `pause muser finis fp` and removes the process; a subsequent completed system `list` no longer contains it. The pause notice itself can be printed before removal finishes; do not use that partial output as the boundary for an immediate raw-memory check. Do not interpret this message as meaning that the process remains available for `start`.

An explicit removal also works: create `new mremove size 100000 run` in `s`, select `s` again, and enter `proc mremove remove`. Wait for `ready`; `list` then no longer contains `mremove`.

Before ending or removing a process

Removal discards its execution state. Export wanted files first, or complete the retained-file procedure in Keep guest files across sessions (p. 18). Do not rely on direct `proc name remove` to clean up every temporary catalog entry: entries can remain visible after removal. For predictable cleanup, use `clear temp` with the relevant file names inside FP before removal, or end the idle FP normally with `finis`. A persistent disk overlay is not a substitute for deliberate guest file retention or cleanup.

5.4. Set the monitor date later

Select `s` and supply all six fields:

Guest system process s: set the date

```
date 86 09 30 08 00 00
```

Historical date commands are different

The historical *RC 4000 Software: Multiprogramming System*, Sections 17.28–17.29, describes `newdate` for setting a day-first date and bare `date` for displaying it. Do not copy those commands into this session: the delivered system uses `date` with six year-first fields to set the clock, and bare `date` is not a date inquiry. The startup year restrictions and host-date translation described above still apply.

5.5. Troubleshooting terminal routing

A queued FP read can delay a system reply

Selecting `s` does not make every outstanding terminal request disappear. The following bounded case is checked with both supported monitor profiles: an ordinary FP named `mtest` is idle after a completed command, attention is requested, `s` is selected, and `list` is entered. Instead of immediately printing the system report, the terminal first prints `to mtest` and waits for input. This is not evidence that `list` was interpreted by FP.

In this specific situation, press Return once to supply an empty line to the idle FP. The queued report then appears:

Guest output: delayed system list (bracketed line is an instruction)

```
to mtest
                        [press Return once here]
from s
mtest         ...       200000   ...   0    wait   s
ready
```

The ellipses stand for session-dependent fields; the bracketed instruction is not text to enter. The checked row's name, core address and size agree with the earlier descriptor inspection. Wait for `from s` and the final `ready` before sending another system command. Sending that command at the interim `to mtest` read can instead give it to FP and produce `***fp name ...`. Attention followed by `mtest` and Return selects the user process again; a subsequent `lookup montest` is verified to work normally.

This empty-line procedure is checked only for an idle ordinary FP at its command input. Do not apply it blindly to a compiler, editor, montest dialog or other program: a blank line can have a different meaning there. This is a terminal-request ordering example, not a general error-recovery rule or a verified multi-process creation/removal recipe.

Two idle FPs sharing one terminal

Two ordinary FPs can share the local console with the default MONITOR/RC433X profile, but input must be directed explicitly to the intended process. For independent interactive work, prefer one local terminal and a Telnet terminal. The shared-console procedure is not established for STDMON/DFC403.

This is an interactive turn-taking procedure, not a pasteable script. Before *every* system command, request attention and select `s` again. The checked test uses the explicit local simulator sequence: configured escape key, `attention`, `continue`, then `s` and Return at `att`. After a system reply's `ready`, the terminal can immediately print `to one` or `to two`; a following system command without reselection would then belong to FP.

The queued-read issue can occur repeatedly, including during another process's output. In the checked case, an explicit `to one` or `to two` requesting an idle FP command is served with an empty Return. Wait for the pending system command's `ready`, or the complete requested utility report, rather than treating an isolated BEL or the first report line as completion. Repeated requests may require repeated empty lines; do not send them speculatively before observing the input context.

Prefer separate terminals for independent interactive work

Use this only with ordinary FPs idle at command input and short utility reports. It is not a recipe for rapid pasted commands, long shared-console reports or arbitrary interactive programs. A blank line can be meaningful to another tool. For everyday independent sessions use separate local and Telnet terminals (p. 98). Removing a process discards its execution state; handle wanted files and temporary-file cleanup first.

6. Files, catalog entries, and Algol programs

Begin with an idle `muser` FP created by Create and inspect a process (p. 10). Skip the optional stop/remove exercise until after this tutorial. If you already ended `muser`, request attention, select `s`, check `list` and create it again with `new muser size 100000 temp disc 200 10 run`. Use a fresh disposable session if the example file names already exist.

These examples use temporary entries and need the native image's `fromhost` and `tohost`. In a second host terminal, change to the emulator's working directory and use a text editor to create the ASCII/LF file `exchange/input` shown below. Ensure that `exchange/output` does not already exist: host exports deliberately refuse to overwrite files.

6.1. Import, copy, rename, and export

For the display and editor examples later in this manual, use exactly these two lines as the contents of host `exchange/input`, including the final LF newline:

Host text file: exchange/input (not a command)

```
Monitor manual sample.
Second line.
```

Enter the following lines in FP, one at a time:

Guest FP in muser: import, copy, rename and export

```
lookup fromhost tohost slang algol
msrc=fromhost input
lookup msrc
mcopy=copy msrc
rename mcopy.moved
lookup moved
output=tohost moved
```

`lookup` displays catalog descriptors for the named entries. The first line locates the installed utilities; `msrc` and `moved` are temporary files in this user process. Their descriptors include allocation, device, date, and scope information. The simulator's offline disk listing is not a substitute for FP `lookup`: it cannot see changes that exist only in a running session's volatile overlay.

`name=command ...` directs the result to an RC catalog entry, except that the left-hand name of `tohost` is the host destination. `copy` copies the text into a new entry. The copy may use fewer disk segments than the source; equal text does not imply identical allocation. `rename old.new` changes the catalog name: the dot between the two names is required. The final line creates host `exchange/output`, containing the original text with LF line endings. Transfer byte counts can include RC padding; they need not equal the host file length.

Avoid naming entries after installed utilities. These are ordinary file operations, not disk initialization: do not use historical catalog creation or reset commands to prepare this session.

Guest names and host names are different

`msrc` and `moved` are guest catalog names. `input` and `output` name host files beneath the configured exchange root. Do not enter `exchange/input` as a guest catalog name or try to use a guest name as a host pathname. If a host export already exists, choose a new output name instead of deleting unrelated files or assuming the export replaced it.

6.2. Display a text file on the guest console

In the same FP process, use the special console destination `c`:

Guest FP in muser: display one text file

```
c=copy msrc
```

For an imported ASCII/LF file containing `Monitor manual sample.` and `Second line.`, the two lines appear on the guest console. A `segm.` copy summary follows the text; it is a utility report, not part of the file. The command returns to FP without attention or a reset. It does not edit the source or create a new disk file named `c`.

To display several text files consecutively, name them in the desired order:

Guest FP in muser: display two text files

```
c=copy msrc moved
```

Since `moved` is the unchanged copy in the earlier example, this displays the same two-line text twice, followed by input and destination summaries. The tested source and copy remain unchanged after both displays.

The destination matters: `result=copy source` creates a disk result, whereas `c=copy source` sends text to the console. Do not assume bare `copy source` is a file-viewing command; it produced only a copy report in the exploratory session. Do not substitute host `cat`: the installed guest `cat` is an entry point in `catsort`, not the host text utility.

This is a text display, not a pager, binary dump or full-screen editor. Large files can scroll past the terminal's history; export them with `tohost` and inspect the host file when necessary. Additional `copy` options, other console destinations and control-character handling remain unverified by this example.

6.3. Compile and run a small Algol program

The optional tutorial bundle includes this example as `examples/hello.alg`:

Algol source: hello.alg (not an FP command)

```
begin
   write(out, <:Hello from Algol<10>:>)
end
```

In a second host terminal, change to the same working directory as the emulator and copy the bundled source. If you did not copy `examples/`, create `exchange/alginput` with exactly the source shown above instead.

Host shell in the emulator's working directory: prepare Algol input

```
cp examples/hello.alg exchange/alginput
```

Then enter these FP commands in the existing `muser` process:

Guest FP in muser: compile and run Algol

```
asrc=fromhost alginput
mprog=algol asrc connect.no
mprog
```

The supplied compiler prints `algol end 24` for this source. Running `mprog` prints `Hello from Algol`, then returns to FP with `end 7`. The end numbers are program reports, not host exit statuses. Compilation and execution both happen on the emulated RC4000, not through the host C compiler.

6.4. Clean up and keep results

If you want to continue with the editor (p. 30) or text comparison (p. 32) examples, keep `msrc` and `moved` until those examples are complete. Skip cleanup for now if you will continue with those recipes; their prerequisites also explain how to recreate the files. The cleanup below deliberately removes both names.

After exporting any wanted results, remove just the example entries in FP:

Guest FP in muser: remove the example entries

```
clear temp msrc moved asrc mprog
```

This deletes those temporary catalog entries. Successful `rename` and `clear` need not print a confirmation. Use `lookup` to inspect entries when unsure, and check the current command context before retrying a command that reports a syntax error or cannot find a program.

Temporary catalog scope and emulator disk persistence are separate concepts. A persistent overlay or writable image does not turn a temporary entry into a permanent file: the guest can reclaim it when its process is removed. For the workflow above, `tohost` is the verified way to keep text beyond the guest session. See the persistent guest-file workflow (p. 18); advanced catalog administration and the full historical utility set are not covered by these examples. Bounded command-file examples are described separately in FP command-file input (p. 36).

7. Keep guest files across sessions

Saving inside the guest and saving the host disk are two separate operations. The workflow below is tested with the default monitor and RC433X interface, using a verified complete working copy of the native image. It does not change the shipped images and does not require an overlay file.

Separate overlay-based restart checks also pass with both maintained-monitor profiles. Text and binary files promoted with `scope user` survive normal FP exit and simulator restart with identical contents and complete lookup metadata. The protected profile uses `rcin`/`rcout` and its own disk/overlay; never reuse the compatibility sidecar for it. Both directions of cross-profile overlay reuse are rejected. This verifies clean restarts, not power-loss recovery or concurrent writers. See Choose a maintained-monitor profile (p. 70) for profile selection and Choose how RC433X disk writes survive (p. 73) for overlay configuration. The write-through recipe below remains a separate compatibility-profile procedure.

Guest entry	Host disk mode	What survives
`temp`	Any mode	The guest can reclaim the entry when its process is removed, even if its sectors have been written to the host filesystem.
`user`	Default volatile overlay	The entry survives process removal in the same machine session, but is lost when the emulator exits.
`user`	Writable working copy	The entry survives process removal and emulator restart on that copy.

7.1. Prepare a writable working copy

This is a separate fresh-session recipe. Export any wanted results and quit an existing emulator before changing its disk mode. In a host shell in your runtime directory (either `build/` or your copied working directory), create a new disk copy and explicitly select it:

Host shell: create and boot a writable working copy

```
mkdir -p work
cp -i UNIT0 work/UNIT0.saved
./sim4000 -a --disk work/UNIT0.saved --disk-write-through
```

Choose a destination that does not already exist. If `cp -i` asks whether to overwrite one, answer no and reuse that existing disk only if it is the working copy you intended; otherwise choose another name and adjust the launch command. For later sessions, do not repeat the copy: start with the saved disk. Never select the shipped `UNIT0` or `UNIT0.original` for write-through mode. Copy only a stopped, complete image. See Configuration and disk persistence (p. 70) for persistent overlays instead; the guest retention steps are the same.

Put a small ASCII text file named `input` in the host directory `exchange` before importing it. A convenient existing text sample can be copied there from the tutorial bundle with `cp examples/hello.alg exchange/input`. Complete the usual date initialization, then press Escape and select `s`.

7.2. Allocate resources and promote a file

Enter this system command in a fresh session:

Guest system process s: create keeper with bounded disk claims

```
new keeper size 100000 temp disc 200 10 perm disc 200 10 run
```

`temp disc 200 10` provides temporary space; `perm disc 200 10` provides a permanent claim of 200 segments and ten entries. These are resource claims, not file-creation commands. Use explicit, bounded claims rather than an unrestricted administrative `all` process for ordinary work.

In the new FP process, enter:

Guest FP in keeper: import and retain the example file

```
saved=fromhost input
lookup saved
scope user saved
lookup saved
```

The first lookup reports `; temp`, with permanence key 0. The second reports `; user`, with permanence key 3. With the supplied default process intervals, the final two base values are both `8388605`. This `scope user` operation is the tested promotion from a disposable import to a retained guest file; no separate `permanent` command is required in this workflow.

Here `user` is a guest catalog scope, not a host login name and not the process name `keeper`. A new process with the same default catalog interval can find the entry. Custom base/login ranges and other scopes can change visibility; the example does not establish access rules for those cases. This is not host or guest security isolation.

Allocation and catalog errors must be resolved before ending the process. Do not infer a successful promotion from a quiet terminal: check the lookup descriptor. Also avoid reusing a name that already describes a retained file. The small imported file can occupy 36 RC segments even when its text is much shorter; permanent resource claims must accommodate the allocated area.

7.3. End the process and read the retained file

Enter `finis` in FP and wait for `pause keeper finis fp`. Select `s` again with Escape and create a different process:

Guest system process s: create reader after keeper has ended

```
new reader size 100000 temp disc 200 10 perm disc 200 10 run
```

In its FP session:

Guest FP in reader: check and export the retained file

```
lookup saved
afterproc=tohost saved
finis
```

The lookup still reports `user`. The host file `exchange/afterproc` contains the original input text. A temporary entry left in `keeper`, by contrast, is reclaimed when that process is removed.

After `pause reader finis fp`, press Control-E and enter `quit` at the simulator prompt. Start a new emulator with the same working copy:

Host shell: restart on the same writable working copy

```
./sim4000 -a --disk work/UNIT0.saved --disk-write-through
```

Initialize the date again, select `s`, and create `reader` with the same resource command. In FP, enter:

Guest FP in reader: check and export after restart

```
lookup saved
rebooted=tohost saved
```

The saved entry is still available, and `exchange/rebooted` contains the same text. Use unused host output names: `tohost` refuses to overwrite an existing export. A repeated session may therefore need a different output name even though reading the guest file succeeds.

7.4. Reclaim the example file deliberately

This cleanup changes the persistent working copy

Skip this section if you want to keep `saved`. Demoting the file to temporary scope and ending its process makes the monitor reclaim it; the write-through copy records that change too. Export or back up wanted contents before following this cleanup. The command names below refer only to the example file, not installed system software.

After exporting any wanted result, demote only the example entry:

Guest FP in reader: deliberately release the retained file

```
scope temp saved
lookup saved
finis
```

The lookup now reports `temp`, with permanence key `0`. Normal process removal then reclaims the entry. This deletion is persistent on the writable working copy; keep a backup or host export if you may want the file later. Do not demote installed utilities or shared libraries as part of this cleanup.

7.5. Inspect catalog information

The protected profile also passes bounded checks of `catstat`, `cata`, bare `show` and `claim`, including the allocation/ reclamation example below. Use `rcin`/`rcout` for its host-file transfers. Compare `catstat` with a catalog export from that same guest session: startup can remove old temporary entries and create

console entries, so an offline listing of the base image need not have the identical temporary contents. These checks do not establish every argument or catalog scope.

Both profiles also pass the `search` and `catsort` scope examples below, including exported reports, temporary-to-user promotion and cleanup. The compatibility image has 414 ordinary system entries; the protected image has 416, including `rcin` and `rcout`. Counts and report totals quoted below refer to the compatibility image unless stated otherwise. In protected sessions replace `fromhost`/`tohost` with `rcin`/`rcout`; the other demonstrated commands are unchanged.

The following are separate tasks, not one script to run after removing `saved`. Use a live ordinary FP with unused report names. Examples referring to `keeper` need the temporary and permanent claims described earlier; examples reading `saved` require that entry still to exist and be visible in the process's catalog interval. Do not repeat an import over an existing retained entry merely to follow the next example.

Question	Context and tool
Inspect a known guest name	Guest FP: `lookup` *name*.
List currently visible retained files	Guest FP: `search user`.
List installed names alphabetically	Guest FP: `catsort scope.system`; inspect the special `catalog` entry separately.
Check free guest disk allocation	Guest FP: bare `show`.
Check free main-catalog slots	Guest FP: bare `cata`; this is not free disk space.
Inspect the process's resource counters	Guest FP: bare `claim`; see its version-specific limitations in the resource example (p. 11).
Inspect stored main-catalog occupancy	Guest FP: bare `catstat`, with IX enabled. This is not a live file-access check.
Inspect an image without booting it	host shell: `--list-catalog` or `--list-main-catalog` options to `sim4000`; stop the writer first.

The names `show` and `claim` above are guest programs, not simulator commands. The offline listers read stored disk records, not the running guest's visible namespace. Choose the tool for the question rather than treating these reports as interchangeable directory listings.

FP `lookup saved algol slang` is the tested way to inspect several named entries in the running guest. To retain that report as guest text and export it, use these also-tested FP commands:

Guest FP: inspect named entries and export the report

```
details=lookup saved algol slang
detailsout=tohost details
```

`details` is a temporary text result, not a live catalog database. Its host export `exchange/detailsout` is readable without a binary decoder. It lists only the names requested, not every file on disk. Bare `lookup` does not provide a complete directory listing in this version.

List a scope with `search`

Use `search`, another entry point in `utility`, to list entries without knowing their names in advance. In an FP process:

Guest FP: list one catalog scope per command

```
search temp
search user
search system
```

These select temporary entries, retained user entries, and installed system entries respectively. They print descriptors like `lookup`, not just names. The selectors are scope names, not disk names: `search disc` is not the way to list the disk. Bare `search` gives no useful listing in the tested version. Use a separate call for each scope; `search temp user system` did not produce a combined listing in the checkpoint.

The system listing is long. To collect it as text instead of scrolling the terminal, use the following workflow in the `keeper` process described above:

Guest FP in keeper: collect and export scope reports

```
templist=search temp
userlist=search user
syslist=search system
rootlist=lookup catalog
listing=copy templist userlist syslist rootlist
listout=tohost listing
```

`copy` concatenates the input text files in the order given. The resulting host `exchange/listout` contains the reports and can be read with a host text editor. Choose unused result and export names. The checkpoint's bounded 200-segment/ten-entry temporary claim is sufficient for the supplied image and this example; larger or substantially changed images may need more space.

The special `lookup catalog` descriptor ends its first line with `; ***`: its unusual interval has no ordinary printable scope label. This inline label is not a leading `***lookup` error diagnostic.

The extra `lookup catalog` is intentional: this image's special `catalog` entry has an upper base of `8388606`, unlike ordinary system entries ending at `8388605`, and is not included by `search system`. The combined export has been checked against all 415 names in the shipped native catalog, including 11-character names that touch `=` without a separating space. It also contains the newly created temporary and user test entries. Labels such as `system.disc` can appear on shared-file entry points; they are descriptor labels, not additional names to type as programs.

This is coverage of the supplied image in a process with the default catalog intervals, not an unrestricted directory scan under every possible base or login arrangement. The calls are separate snapshots, not an atomic combined view: a temporary report can include its own result entry and a short-lived `wrk` working entry that has already disappeared when the next command runs. Reports created later will not appear in an earlier snapshot. Do not expect alphabetical ordering. After exporting, remove the report files with:

Guest FP: remove only the temporary listing reports

```
clear temp templist userlist syslist rootlist listing
```

Normal `finis` also reclaims these temporary reports. No promotion to `user` is needed merely to export a listing.

Alphabetical catalog reports with `catsort`

For a compact, alphabetically sorted guest report, enter in an ordinary FP process with console output selected:

Guest FP: print compact catalog reports

```
catsort
catsort scope.system
```

Bare `catsort` reports local catalog groups, not just temporary files. In a freshly created `muser` process with no retained user files, it lists only the temporary console entries `c` and `v`, with a total of zero segments and two entries. It is not a complete disk directory. The scope-selection example below shows what changes when a user file exists.

`catsort scope.system` lists the installed system entries. On the supplied image, the tested report contains 414 different names in alphabetical order, including `algol`, `fromhost`, `slang` and `tohost`. Like `search system`, it excludes the special `catalog` entry; inspect that separately with `lookup catalog`. The report has page headings, base/scope information, entry sizes or type markers, document names and encoded descriptor metadata. For an unambiguous named descriptor, use `lookup name`; not every numeric field or type marker in the compact report is decoded here.

The native-monitor report has eight numbered page headings; the tested standard-monitor variant has seven. Both use form-feed characters between pages. Neither pauses for a key at each page: use terminal scrollback or export the report as described below. The final entry total is 2,818 segments and 414 entries; a separate disk summary prints 967 slices, 2,901 segments and 414 entries. These are catalog report totals, **not free disk space** and not the process's remaining resource claim. Use `show` and `claim` for those questions.

Both commands return to FP without attention, and the regression checkpoint runs them in a volatile disk overlay without changing the shipped image. Selectors beyond the examples here, custom sort keys and the `cat` alias have not been verified. `cat` is a guest entry point in `catsort`, not the host file-display command; use `c=copy filename` to display guest text.

To keep the alphabetical system report as text, choose unused report and host export names and enter:

Guest FP: save the system report, then restore console output

```
o sortreport
catsort scope.system
o c
sortout=tohost sortreport
lookup sortreport
clear temp sortreport
```

`o sortreport` selects a temporary guest text file as FP's output destination. The report goes there instead of to the console; command echoes may still appear on the console. `o c` restores console output before export or cleanup. The host file `exchange/sortout` contains all 414 names and the same descriptor rows and totals as the console report. Text export removes guest text padding and the end marker and uses LF line endings; form feeds remain page separators. The example's 200-segment/ten-entry temporary claim is sufficient for this shipped image. The report is a snapshot, not a live directory database.

After export, `clear temp sortreport` removes only the guest report, not the host file. The regression checkpoint confirms that the name becomes unknown and guest free disk space returns to its pre-report value. If export fails, keep the report until the cause has been corrected; do not clear it merely because a later FP command can still run.

Distinguish temporary and retained files in `catsort`

This scope demonstration is independent of the earlier retention-and-cleanup recipe. Start a fresh disposable session, initialize the date, select `s`, and create `keeper` with `new keeper size 100000 temp disc 200 10 perm disc 200 10 run`. Prepare a short ASCII/LF `exchange/input` file in the host working directory. In that idle FP, import two small files under unused names:

Guest FP in keeper: compare temporary and retained scopes

```
saved=fromhost input
gone=fromhost input
catsort scope.temp
scope user saved
catsort
catsort scope.temp
catsort scope.user
```

Before promotion, `catsort scope.temp` lists `c`, `gone`, `saved`, `v` in alphabetical order. After promotion it lists only `c`, `gone`, `v`: the retained `saved` entry is no longer temporary. The two imports in the checkpoint occupy 36 segments each; the temporary total drops from 72 segments/four entries to 36 segments/three entries.

The other two reports are broader. Bare `catsort` first lists `saved` under a `project` heading, then `c`, `gone`, `v` under `temp`. Explicit `catsort scope.user` lists the same names, but labels the first group `perm`. Both groups have the base pair `8388605 8388605` in this supplied system. The `perm`/`project` report labels do not replace the `; user` descriptor reported by `lookup saved`. Sorting is within each group, not necessarily across the whole report.

Consequently, **`catsort scope.user` is not an exact equivalent of `search user`** and is not a retained-files-only filter in this environment. Use `search user` when that distinction matters, and inspect scope headings rather than assuming every row has the scope named in the command. These observations apply to the supplied native monitor and default process intervals, not every historical `catsort` version or login arrangement.

These reports do not change the two files. The checkpoint exports `saved` unchanged after process removal and after a fresh machine boot on a writable working image. The temporary `gone` is reclaimed when its process ends. Actual retention still requires a persistent overlay or writable working copy; `scope user` alone cannot preserve a volatile overlay across simulator exit. Use the earlier persistent-file workflow for deliberate removal of `saved`, and never demote installed utilities for this example.

Statistics and offline inspection

At FP, the installed guest program `show` reports free disk allocation:

Guest FP: ask for free guest disk space

```
show
```

For example, in the checked session before importing files it prints:

Guest output: free-space report (session-dependent values)

```
free disc space:
disc        :  1025 slices,   3075 segm.
```

These numbers change with the files currently allocated. The supplied `disc` uses three 768-byte RC segments per slice; a slice is therefore 2304 bytes. The report concerns guest disk space, not available host filesystem space, unused catalog entries or the current process's remaining resource claim. The example process still needs its `temp disc 200 10` claim even when `show` reports more than 200 free segments.

For a bounded allocation/reclamation example, put the two-line ASCII/LF sample used by the editor in host file `exchange/input`, choose an unused guest name `spacetmp`, and enter in FP:

Guest FP: check bounded allocation and reclamation

```
show
spacetmp=fromhost input
lookup spacetmp
show
clear temp spacetmp
lookup spacetmp
show
```

The checked import allocates 36 segments, as its `lookup` descriptor shows. Free space decreases by 12 slices / 36 segments, not merely by the short text's byte length. After `clear`, `lookup` reports `***lookup spacetmp unknown`, and `show` returns to the original free-space counts. That unknown-name diagnostic is expected after deletion. Repeated bare `show` calls without intervening allocation give unchanged counts. This workflow returns to FP without attention or a reset and works with both supported monitor/controller configurations. Other `show` arguments, multiple disk devices and fragmentation reporting remain unverified.

Do not confuse this guest command with `show cpu`, `show media` and the other simulator commands at `sim4000>`. In particular, do not add a filename to bare guest `show` expecting to print its contents.

The existing `cata` program runs without arguments and prints Danish catalog statistics, including:

- `antal segmenter`: number of segments;
- `indgange ialt`: total entry slots;
- `frie indgange`: free entry slots.

It also prints occupancy/key-frequency distributions. It is not a file-name lister. These figures describe the monitor's main catalog area `catalog`, not the device catalog `catdisc`. The two stored layers in native `UNIT0` differ:

Layer	Segments	Occupied entries	Offline command
Device catalog `catdisc`	36	415	`--list-catalog`
Monitor main catalog `catalog`	72	418	`--list-main-catalog`

Both use 15 entry slots per segment: 1080 main-catalog slots versus 540 device slots. The pristine main area contains the same names plus `c`, `v`, and `catdisc`; its area document fields use `disc` where device-level records can use `wrk000011`. These are not 418 separate applications. `cata`'s free-slot report has been checked against empty records in a binary export of `catalog` during the same session. It is not a free-disk-space report.

For a complete read-only name and metadata list after leaving the writable session, run in the host shell:

Host shell: inspect a stopped session's working disk image

```
./sim4000 --list-catalog work/UNIT0.saved
./sim4000 --list-main-catalog work/UNIT0.saved
```

These commands see stored image records, not a running emulator's volatile overlay or in-memory catalog state. The main area can contain obsolete temporary descriptors: the checked working image still contained `gone` there after its file had disappeared from the device catalog. A listed descriptor is not proof of accessible backing storage. Use guest `search`/`lookup` for the running session; neither offline command is a complete consistency checker. Do not read during guest writes.

Stored catalog occupancy with optional IX

The supplied `catstat` program needs the IX index instruction, which is enabled by default. Start the emulator with `./sim4000 -a`, then enter this FP command:

Guest FP with IX enabled: report stored catalog occupancy

```
catstat
```

This bare invocation lists names and occupied-entry counts for all 72 stored main-catalog segments (418 stored records in the supplied image), then returns normally to FP with `end 18`. The output agrees with the offline main-catalog listing, including duplicate names and their segment positions. Stored records are not necessarily accessible files: this is not a live namespace check or a free-capacity report. Use guest `search`/`lookup` to check current accessibility. Other `catstat` argument forms have not been validated with IX enabled.

IX is enabled by default, also when the configuration omits `CPU IX`. `CPU IX=NO` disables it; `--no-ix` overrides an enabling configuration; `show cpu` reports the selected capability. This is a narrow compatibility extension, not a complete RC8000 CPU profile or a replacement monitor. IX follows Henrik Jacobsen's newer emulator source; its precise historical hardware-model assignment has not yet been verified. It is an instruction, not an emulated CPU flag. `CPU IX=YES` is the emulator's capability switch.

IX-disabled diagnostics are not the normal workflow

With IX enabled (the default), the tested bare `catstat` completes. The specific recovery example below deliberately disabled IX; it is not part of default startup or normal catalog inspection. There is no need to provoke a guest break merely to inspect the catalog.

Leave IX enabled when using `catstat`. With IX disabled it pauses with `break 0 line 18-19`; changing the input file is not a working workaround. Use `search` or `catsort` instead if that extension is deliberately disabled.

Recover FP after this guest program break

The failed `catstat sortreport` investigation with IX disabled produces the following output. This is a recovery example for that specific paused process, not a required step in normal catalog inspection:

Guest output: the investigated IX-disabled program break

```
break        0  line  18-19

from s
pause muser break fp
```

The simulator is still running; it is the guest process that has paused. For this tested case, press bare Escape, select `s` at `att`, then enter:

Guest system process s: reinitialize the paused FP

```
proc muser start
```

Replace `muser` with the actual paused process name. The supplied system prints `***fp reinitialized` and accepts FP commands again. This is a reinitialization of the file processor, not a continuation of the failed `catstat` computation. A complete emulator restart is not needed here.

After this specific recovery, use lookup to inspect any wanted files and export them under unused host names before cleanup. Export important files before deleting or removing their process; do not use this example as a reason to assume all resources survive every kind of guest failure.

The recovery is checked with both supplied monitor/controller variants for this specific break. Other traps, damaged FP state and interrupted command files have not been validated. A guest Algol `break` message is distinct from the simulator's execution breakpoints and its Ctrl-E operator prompt.

8. Software and files in the supplied disk image

Henrik Jacobsen describes most basic software on UNIT0, apart from the monitor, as later RC8000/RC9000-10 versions. This is a mixed historical software environment, not a pristine 1969 RC4000 installation. That lineage does not imply RC8000/RC9000-10 hardware emulation; the descriptions below distinguish identified entries from workflows actually tested here.

The native `UNIT0` is a historical working system, not an empty disk or a collection of 415 independent applications. Its 415 occupied catalog entries include programs, text and data, libraries, device descriptions, and named entry points into shared files. This section describes the entries most relevant to an operator or someone writing guest programs. It is not a promise that every installed package works with the native emulator.

The development examples use the optional `examples/` directory beside the runtime files. All host copy commands below are entered in a second host terminal in that same working directory. See Runtime files and tutorial inputs (p. 5) if the directory is missing. These are separate recipes: begin each with an idle FP, unused guest names and unused host export names. Cleanup for one recipe must not remove inputs you still need for another.

The descriptions use three evidence levels:

- **Tested:** the stated workflow has run successfully in Sim4000. This does not certify every option of that program.
- **Documented:** historical sources or inspected file contents establish the role; the complete workflow has not been tested here.
- **Inventory only:** the entry exists, but its exact operation is not yet established. A suggestive name is not a verified interface.

Unless stated otherwise, program calls belong in FP inside an ordinary user process, not at `sim4000>` or directly in `s`. Start with the process and temporary disk claim described in Processes and command contexts (p. 10). Use lower-case names as shown in the catalog.

8.1. Everyday commands and development tools

Entry	Purpose and use	Evidence
`fp`	File processor: the guest command interpreter started in an ordinary process. It dispatches utility and program calls. Normally use it through `new ... run`, not by typing `fp` again.	Tested
`lookup`	Inspect named catalog entries, for example `lookup algol slang`.	Tested

Entry	Purpose and use	Evidence
`copy`	Copy or concatenate text into a result entry, or display it with `c=copy source`; see the file examples above.	Tested text copy, concatenation and console display
`rename`	Rename an entry with `rename old.new`; the dot is significant.	Tested
`clear`	Remove named entries, for example `clear temp duplicate`. Do not use installed global utility names as cleanup targets.	Tested for named temporary entries
`scope`	Promote a named temporary file to `user`, or demote it to `temp`; see the persistent-file workflow above.	Tested for these two scopes
`search`	List descriptors by scope, for example `search user`. Separate calls for `temp`, `user`, and `system` can be collected as text.	Tested for default intervals
`catsort`	Compact catalog reports, sorted within scope groups: `scope.temp` for temporary entries, `scope.system` for installed entries; bare and `scope.user` can include several groups.	Tested local scope groups, complete system-name coverage, totals and export
`cata`	Print main-catalog slot statistics, not a name listing or free disk capacity. Stored-record limitations are described above.	Tested slot accounting
`show`	Report free guest disk space in slices and RC segments. This is not the simulator's `show` command.	Tested bare call, allocation decrease and reclamation
`claim`	Report guest process/core and disk resource counters; see the version-specific limitations above.	Tested bare report and bounded temporary allocation/release
`edit`	Command-driven text editor, called as `result=edit source`; see the examples below.	Tested search/replacement, line movement, deletion, insertion and completion
`cmp`	Compare two text entries with `cmp first second`; report identical text or line differences.	Compatibility: identical text, replacement and export tested; protected: tofrom1 break
`fromhost`, `tohost`	Import from and export to the configured host exchange directory. Text and binary transfer are supported.	Tested

Entry	Purpose and use	Evidence
`algol`	Compile Algol source into a guest executable, for example `program=algol source connect. no`. See the small program above.	Tested
`slang`	SLANG assembler, called as `program=slang source`. It assembles RC machine code inside the guest.	Tested
`i`, `o`	Read FP commands from a text entry and select FP's output destination. `o c` restores console output; see the command-file example below.	Tested for one input file and listing output
`if`	Conditionally execute the next command or parenthesized group using `if ok.yes` / `if ok.no`.	Tested after successful and failed `lookup`, including grouped commands
`finis`	End FP normally. In the supplied system this also causes `s` to remove the ordinary user process and reclaim its temporary resources.	Tested

Several of these names, including `edit`, `lookup`, and `copy`, are entry points in the shared `utility` file, not separate allocated program files. Keep `utility` even if you mainly use the compiler. The historical distribution identifies this System Utility as an RC8000 version from 1987: this image contains software from several periods, not just the original 1969 system.

8.2. Edit a text file inside the guest

The command-driven `edit` operations below have also been checked in the protected profile: replacement, insertion/deletion, truncation and reopening an edited guest file all produce the expected exported bytes without changing the originals. For that profile substitute `rcin`/`rcout` for `fromhost`/`tohost`, and use `exchange/` for host files. This is not a full-screen editor or a test of every historical editor operation; see Choose a maintained-monitor profile (p. 70).

The editor has its own input language

Enter `result=edit source` in FP; subsequent editing commands go to `edit`, not FP. Its `f` operation completes the result and returns to FP. It is not `finis`, which ends the FP process. Only export or clear the result after the editor has finished. Control-E suspends the emulator; it does not finish or save an unfinished edit.

After importing a file with `msrc=fromhost input`, where the first line contains `Monitor manual sample.`, enter:

Guest FP: open the source in edit

```
medit=edit msrc
```

Guest editor: locate, replace and finish

```
l./Monitor/,r/Monitor/Edited/,f
```

Guest FP, after the editor finishes: export the result

```
edited=tohost medit
```

The second line is an editor command line, not an FP command. `l./Monitor/` locates the text, `r/Monitor/Edited/` replaces it, and `f` finishes the edit and copies the remaining input to the result. Commas separate editor operations. The resulting host `exchange/edited` contains `Edited manual sample.` on its first line and preserves the remaining text. Choose a host destination that does not already exist.

This creates a separate temporary result; it does not overwrite `msrc`. Keep `medit` for the comparison example that follows. After that example, `clear temp medit` removes the result. Do not interrupt an unfinished edit and assume it has been saved. This editor has no modern full-screen cursor interface, and ordinary terminal editing shortcuts are not a substitute for its command language.

For a tested multiline example, copy `examples/editor_lines.txt` to host file `exchange/edinput`. From the emulator's working directory this is:

Host shell: copy the multiline input from the emulator's working directory

```
cp examples/editor_lines.txt exchange/edinput
```

The input contains four lines: `First line.`, `Keep this line.`, `Remove this line.`, and `Last line.`. Enter the following in FP, including the standalone `!` line:

Guest FP: import and start the multiline edit

```
mlines=fromhost edinput
mpatch=edit mlines
```

Guest editor: move, delete, insert text and finish

```
l1,l1,d0,i!
Inserted line.
Literal comma, slash / preserved.
!
f
```

Guest FP, after the editor finishes: export the result

```
edops=tohost mpatch
```

In this example, each `l1` advances one line, retaining the passed text. After two advances, `d0` deletes the current `Remove this line.` line. `i!` begins insertion; `!` is the chosen closing delimiter and must not occur in the inserted text. While insertion is active, the comma and slash above are literal text, not editor operations. `f` completes the result, including the untouched last line. The host file `exchange/edops` contains:

Host text file: exchange/edops (expected output)

```
First line.
Keep this line.
Inserted line.
Literal comma, slash / preserved.
Last line.
```

Deletion counts are easy to misread: the tested `l2,d1,f` sequence on the same four-line input retains only the first two lines, deleting both the current third line and the following fourth line. Use `d0`, not `d1`, for the single-line deletion shown above. These positive movements and deletions are tested; arbitrary backward movement and search-based deletion ranges are not covered by these examples.

The original `mlines` remains byte-for-byte unchanged. Keep/export any wanted result before cleaning up with `clear temp mlines mpatch`. The example works with both supported monitor/controller configurations and does not require changing the shipped image. Historical monitor build scripts in `monsrc/hjmops` contain further editor examples, but are not beginner command files to execute.

The following is a quick reference to the operations tested above, not the editor's complete command language:

Editor input	Meaning in the checked example
`l./Monitor/`	Locate the matching text for replacement.
`r/Monitor/Edited/`	Replace that text in the separate result.
`l1`	Advance one line while retaining the passed text.
`d0`	Delete the current line; `d1` also deletes the following line in the tested four-line input.
`i!`, text, then `!`	Insert text using the chosen closing delimiter. Enter the closing `!` on its own line as shown; commas and slashes within insertion are literal text.
`f`	Finish, copy the remaining input and return to FP.

The separate historical **Screen Editor** should not be confused with `edit`: the original emulator's distribution notes say it could not run because the guest terminal process lacked required facilities. Its operation in Sim4000 has not been established. An ANSI/VT100 connection alone does not establish that compatibility.

8.3. Compare text files inside the guest

Use the compatibility profile for guest cmp

The installed `cmp` does not work in the protected profile: the tested identical-text call pauses with `break 0 tofrom1`. The successful comparison examples below apply to the compatibility profile only. To compare protected guest text, export each entry with `rcout` under different unused host names and use a host text-comparison tool. Text display, copying, renaming and concatenation have been checked in both profiles.
For this specific protected `cmp` break, Escape, `s`, then `proc` *processname* `start` reinitializes FP; substitute the paused FP's actual name. The checked recovery preserves the compared entries for export, but does not resume `cmp`. Inspect and export wanted files before cleanup. Do not assume this recovers every guest fault.

After the file-copy tutorial and the preceding editing example, the original is `msrc`, its unchanged renamed copy is `moved`, and the edited result is `medit`. Run this comparison before their cleanup commands. If those entries have already been removed, recreate them in an FP process with unused names:

Guest FP: prepare the comparison files and start edit

```
msrc=fromhost input
mcopy=copy msrc
```

```
rename mcopy.moved
medit=edit msrc
```

Guest editor: replace text and finish the result

```
l./Monitor/,r/Monitor/Edited/,f
```

For the exact output shown here, host `exchange/input` must contain this ASCII/LF text, including the final newline:

Host text file: exchange/input (not a command)

```
Monitor manual sample.
Second line.
```

At FP, enter:

Guest FP: compare original, copied and edited text

```
cmp msrc moved
cmp msrc medit
```

The first call prints `files are identical`. The second reports:

Guest output: text comparison report

```
DISCREPANCY NO. 1  msrc line 1, medit line 1  TEXT REPLACED
==========>> msrc:
Monitor manual sample.
==========>> medit:
Edited manual sample.
 1 discrepancy found
```

Both calls return to FP without attention, a reset or a new process. A reported difference is a comparison result, not a failed FP command. The calls leave all three text entries unchanged.

To save the report instead of displaying it, choose unused guest and host names and use FP output redirection:

Guest FP: save the comparison report and restore console output

```
o cmpreport
cmp msrc medit
o c
cmpout=tohost cmpreport
clear temp cmpreport
```

The exported ASCII/LF file is `exchange/cmpout` with the default host-files configuration. `o c` restores console output before the export. The cleanup removes only the guest report, not the compared files or the host export.

This is a guest **text** comparison, not a byte-for-byte disk-area or binary comparison: identical text was verified even when the imported source and its copied result had different allocated segment counts. Binary comparison, additional options and more complex difference alignment remain unverified. Use host `cmp` on binary host exports when exact bytes matter.

Compare protected guest text on the host

Once the three sample entries above exist, export each under a different, unused host name. In a protected session, prepare the original with `msrc=rcin input` rather than `fromhost`; the copy, rename and editor commands are unchanged. Then enter at FP:

Protected FP: export the three comparison samples

```
cmporig=rcout msrc
cmpcopy=rcout moved
cmpedit=rcout medit
```

With the supplied protected configuration, run these commands in a host shell from the runtime directory, not at FP or at the simulator prompt:

Host shell: compare exported text

```
cmp -s exchange/cmporig exchange/cmpcopy
echo $?
cmp -s exchange/cmporig exchange/cmpedit
echo $?
diff -u exchange/cmporig exchange/cmpedit
```

For the exact sample text above, the two printed status codes are 0 (equal) and 1 (different). The unified diff shows the first line changing from `Monitor manual sample.` to `Edited manual sample.`; the second line is unchanged. Both tools use a status greater than 1 for an error, such as an unreadable or missing file. Do not treat an error as an ordinary difference. The paths depend on `HOSTFILES ROOT`; compatibility exports normally live in `exchange/` instead. No Python or extra simulator tool is needed for this workflow.

Host comparison is read-only and works after simulator exit because the exports are independent host files. It compares exported text, not guest allocation or untouched binary data. Check each transfer before comparing. If a destination already exists, export is deliberately refused; choose a new host name. The checked refusal leaves the old export's contents, identity, size, mode and modification time unchanged, and FP remains usable.

8.4. Assemble a SLANG source

Copy `examples/slang_integer.sl` to host file `exchange/slinput`, then enter in FP:

Guest FP: assemble and export the valid SLANG example

```
slsrc=fromhost slinput
slprog=slang slsrc
slbinary=tohost conv.bin slprog
```

This supplied example assembles instructions that add 123 and 321, then loop. It is a machine-code demonstration, not a program that returns normally. If you run it with `slprog`, use Control-E to suspend the machine afterward. The binary export must use `conv.bin`, not text conversion. Its backing-store segment contains allocation padding beyond the actual instructions; it is not a host executable. See Host file exchange (p. 50) for conversion modes.

Check the assembly result before using it

The example above uses a known-valid supplied source. For changed sources, inspect the final SLANG report before running or exporting the result as a successful program. A failed assembly can leave an entry marked executable; `lookup` alone is not a success check. See assembly error recovery (p. 41) and the tested export guard (p. 39) before automating compilation jobs.

8.5. Save an assembler listing and use an FP command file

The listing-and-command-file workflow below is checked in both maintained monitor profiles. For protected operation, use `rcin`/`rcout` for actual transfers, including the two `=tohost` lines inside your copy of `slang_commands.fp`. Keep the supplied example unchanged. The protected configuration normally places host files under `exchange/`. Both profiles preserve the source, export the expected listing/code and return to keyboard input. The optional execution recipe below also checks this sample's loop, not every SLANG option or program.

FP can read commands from a guest text entry instead of the keyboard. This is useful for repeatable compilation jobs; it is not a host shell script or a new monitor variant. The following example is tested with both the supplied monitor/RC433X and the standard monitor/DFC403 configurations.

Starting in the emulator's working directory, copy both supplied examples from a host shell:

Host shell: prepare source and FP command-file inputs

```
cp examples/slang_integer.sl exchange/slinput
cp examples/slang_commands.fp exchange/cmdinput
```

Use an ordinary FP process with temporary disk resources, such as the `muser` process introduced earlier. If you have already assembled the preceding example, export any wanted results and remove its temporary `slsrc`/`slprog` entries, or use a fresh process. These examples use the same guest names. Choose unused host destination names `slbinary` and `sllisting` inside the configured exchange directory; host export refuses to overwrite existing files.

The command file contains these five FP lines:

FP command-file contents: slang_commands.fp

```
o slist
slprog=slang slsrc list.yes
o c
slbinary=tohost conv.bin slprog
sllisting=tohost slist
```

Import the source and commands, then select the command input in FP:

Guest FP: import the source and execute the command file

```
slsrc=fromhost slinput
slcmd=fromhost cmdinput
i slcmd
lookup slprog
```

`i slcmd` reads and executes the lines in `slcmd`; after this file ends, FP accepts keyboard commands again. It does not end the process. The subsequent `lookup slprog` is a keyboard command, not part of the file.

`o slist` selects a guest text output entry. `list.yes` makes SLANG write its source listing there as well as its assembly report. `o c` restores output to the RC console before the export commands. Here `c` means the guest console, not the simulator's `continue` abbreviation. While output is redirected, a quiet console does not mean that the assembler has stopped.

The host file `exchange/sllisting` contains the listed instructions and `slang ok 1/12/1`: one SLANG segment, 12 RC bytes and one backing-store segment. An RC byte is a 12-bit halfword, not an eight-bit host byte. The six-word code therefore occupies 18 host bytes at the start of the 768-byte binary export; remaining bytes are allocation padding, not additional sample instructions. `lookup slprog` reports executable content `2.4` (entry offset 4) and length 12.

Optionally run and interrupt the integer sample

Before clearing the successful `slprog`, you can enter `slprog` at FP. This known sample adds 123 and 321 and then loops; it never prints a result or returns normally. Press Control-E and, at the simulator prompt, enter `step 1000000`, then `show cpu`. The checked result is `W0=0001bc` (444) and `W1=000141` (321). The instruction counter points at the `jl. 0` self-loop; its absolute address depends on the process allocation. This bounded debugger step changes execution state, not just the display. With protection selected, the observed execution is in task mode with PR `bf` and storage key 1. This example is checked in both profiles, not a general test of machine-code programs.

To return to FP, enter simulator `continue`, then press bare Escape, select guest system process `s`, and enter:

Guest system *s*: interrupt the sample in muser

```
proc muser break
```

Replace `muser` with the actual process name. Expect `***break 8` followed by the decimal loop address and `pause muser break fp`. Request attention again with Escape, select `s`, and enter `proc muser start`. The reply `***fp reinitialized` means FP has restarted, not that the sample loop has returned. A subsequent `lookup slprog` and the cleanup below work in the checked session. Guest `proc name break` is distinct from a simulator execution breakpoint, Ctrl-E suspension and `proc name stop`. Do not assume every arbitrary machine-code program has the same interrupt recovery behavior.

After inspecting the exports and finishing any optional execution, remove just these temporary guest entries in FP:

Guest FP: remove the assembly example entries

```
clear temp slsrc slcmd slist slprog
```

Command files execute guest commands and can change guest files or processes. Review their contents first; do not execute installed subsystem startup scripts merely to inspect them. This example checks one command input file, output restoration and assembly, not arbitrary nested inputs, conditional commands or batch administration. `slangxref` is installed, but its calling convention and cross-reference workflow remain unverified (inventory only).

8.6. Call one command file from another

The listings labelled *command-file contents* are file contents, not additional keyboard commands. Prepare and import the files first; type only the *Guest FP* steps at the keyboard. The commands following an `i` invocation are keyboard steps after that file has reached EOF and returned input to the console.

A bounded nested-input example is supplied as `examples/nested_outer.fp` and `nested_inner.fp`. From the emulator's working directory, copy them in a host shell:

Host shell: prepare the nested input files

```
cp examples/nested_outer.fp exchange/outerinput
cp examples/nested_inner.fp exchange/innerinput
```

The outer command file contains:

Outer command-file contents: nested_outer.fp

```
o nestlog
lookup fromhost
i innercmd
lookup algol
o c
nestedout=tohost nestlog
```

The inner command file contains just:

Inner command-file contents: nested_inner.fp

```
lookup tohost
```

In an ordinary FP process with the earlier temporary resource claim, choose unused guest names `outercmd`, `innercmd`, `nestlog` and an unused host export name `nestedout`. Import both files **before** starting the outer file:

Guest FP: import both command files and start the outer file

```
outercmd=fromhost outerinput
innercmd=fromhost innerinput
i outercmd
lookup slang
```

At the inner file's end, FP resumes the outer file after `i innercmd` and executes `lookup algol`. The host file `exchange/nestedout` contains the three catalog descriptors in the order `fromhost`, `tohost`, `algol`. All three are in the redirected report, not on the console. The outer file explicitly restores console output with `o c` before exporting; do not rely on input-file changes to restore output automatically.

At the outer file's end, keyboard input resumes without ending the process. The final `lookup slang` above is typed at the keyboard and displays its descriptor on the console. After checking the export, clean up:

Guest FP: remove the nested-input example entries

```
clear temp outercmd innercmd nestlog
```

This verifies one inner-file call and normal end-of-file returns with both supported monitor/controller configurations. It does not establish arbitrary nesting depth, recursion, early input termination, arbitrary conditional commands or general recovery from errors inside a command file. Keep command files small, review their contents, and do not use this example as a general unattended batch-job or subsystem-startup guarantee.

The protected profile also passes this nested-input example and the bounded missing-program and conditional assembly examples below. In protected sessions, use `rcin` for imports and `rcout` for exports.

Review command-file contents as well as keyboard commands: change executable `=tohost` transfers to `=rcout` in your copies. A mere `lookup tohost` still inspects an installed descriptor and need not be changed. Keep the supplied example files intact. These checks do not establish arbitrary nesting or safe unattended jobs.

8.7. Execute commands conditionally

The installed startup files `temtrim`, `primotrim` and `sostrim` use `if ok.no`; their presence alone is not a reason to run those generation scripts. A small independent example verifies the following FP forms:

- `if ok.yes`: execute the next command when the preceding result is successful;
- `if ok.no`: execute it when the preceding result is unsuccessful.

For example, a command file can contain:

Command-file contents: condition on one successful lookup

```
lookup slang
if ok.yes
lookup algol
```

Both descriptors appear. Changing the condition to `if ok.no` skips the `lookup algol` command because `lookup slang` succeeded. A lookup of a nonexistent entry such as `absentone` instead reports `***lookup absentone unknown`; the following `if ok.no` selects its next command. The error does not itself stop the file.

Use parentheses when the condition should control several commands:

Command-file contents: condition on a group of commands

```
lookup slang
if ok.yes
(
lookup algol
lookup fromhost
)
lookup tohost
```

The two grouped lookups run after the successful lookup. With `if ok.no` the whole group is skipped, but `lookup tohost` outside the group still runs. Without a group, the condition controls only the next command, not every later line. The parentheses here are FP syntax, not a host subshell.

For the complete tested example, copy `examples/conditional_commands.fp` to host file `exchange/condinput`, then use an unused guest name in an ordinary FP process:

Guest FP: run and remove the conditional example

```
condcmd=fromhost condinput
i condcmd
lookup slang
clear temp condcmd
```

This example uses both conditions after success and failure, with executed and skipped groups. Expect two `absentone` lookup errors, but no `nosuchprog` name error: those guarded calls are skipped. At EOF keyboard input resumes. These forms are checked with both supported monitor profiles.

These examples establish `lookup` result tests, not that every guest program's printed diagnostics set `ok` in the same way. The supplied SLANG assembler's two checked cases are described next; other program status conventions remain unverified. Numeric `mode` bits, loops, nested conditionals and a general unattended batch policy remain unverified.

Guard a SLANG binary export with its result status

The supplied SLANG assembler sets an unsuccessful FP result for the tested undeclared-identifier error and a successful result for the valid integer sample. This permits a conditional export without executing either program. The complete example is `examples/assembly_status.fp`:

Command-file contents: assembly_status.fp

```
checkbad=slang checksrc
if ok.yes
badcheck=tohost conv.bin checkbad
if ok.no
lookup fromhost
lookup checkbad
clear temp checkbad
checkgood=slang checkvalid
if ok.no
nosuchprog
if ok.yes
checkbin=tohost conv.bin checkgood
lookup checkgood
clear temp checkgood
lookup slang
```

From the emulator's working directory, prepare its inputs in a host shell:

Host shell: prepare the conditional assembly inputs

```
cp examples/slang_error.sl exchange/badinput
cp examples/slang_integer.sl exchange/slinput
cp examples/assembly_status.fp exchange/statusin
```

Select console output (`o c`), choose unused guest names from the example, and ensure host `exchange/badcheck` and `exchange/checkbin` do not exist. In an ordinary FP process with the earlier temporary resource claim, enter:

Guest FP: run and remove the assembly-status example

```
checksrc=fromhost badinput
checkvalid=fromhost slinput
checkcmd=fromhost statusin
i checkcmd
lookup checkbad checkgood
clear temp checksrc checkvalid checkcmd
```

The failed assembly reports `***slang sorry 1 1/4/1`. Its export is skipped: no host file `badcheck` is created. The failure branch prints the `fromhost` descriptor, and the partial result is inspected and explicitly removed. The valid assembly reports `slang ok 1/12/1`; its failure branch skips `nosuchprog`, and its success branch exports the expected instruction bytes to `exchange/checkbin`. The file then explicitly removes the valid result too, so the keyboard lookup reports both result names unknown.

Test the intended result, not a later command

Keep a result-status test immediately after the operation being tested. An intervening completed utility call such as `lookup` can supply a new result status; do not test its success and mistake that for assembly success. In this fixture, skipped conditional commands preserve the status needed by the following complementary condition. All cleanup is explicit, not a transaction or automatic rollback.

Both monitor/controller variants are checked, but other SLANG error classes, other assembler versions and other compilers are not covered. Continue to inspect the assembly report and never run an unverified partial result.

A missing program does not abort a command file

A command file is not a stop-on-error job

In the supplied FP, an unknown program name reports an error but the remaining commands in the input file still run. Do not treat this as shell-style stop-on-error behavior, or assume later cleanup/export commands were skipped. Conditional execution must be written explicitly; see conditional FP commands (p. 38) and guarding a SLANG export (p. 39) for the bounded cases checked here.

The harmless `examples/command_error.fp` example contains:

Command-file contents: command_error.fp

```
lookup fromhost
nosuchprog
lookup tohost
```

Copy it to host file `exchange/errinput`, then import and run it at FP using an unused guest name:

Guest FP: run the missing-program example

```
errcmd=fromhost errinput
i errcmd
lookup slang
```

The console shows the `fromhost` descriptor, `***fp name nosuchprog`, then the `tohost` descriptor. After EOF, the keyboard `lookup slang` works in the same process without attention or reset. The error is deliberately part of this example; no program named `nosuchprog` should be installed.

The same continuation occurs with one outer command file. The supplied `examples/command_error_outer.fp` contains:

Outer command-file contents: command_error_outer.fp

```
lookup slang
i errcmd
lookup algol
```

Copy that file to host file `exchange/errtopin`, keep the imported `errcmd`, and enter:

Guest FP: run and remove the nested error example

```
errtop=fromhost errtopin
i errtop
lookup slang
clear temp errcmd errtop
```

Both the inner `lookup tohost` after the error and the outer `lookup algol` after inner EOF execute. The final keyboard lookup and cleanup still work. This specific missing-program case is verified with both supported monitor variants. It is not a promise that compiler errors, syntax errors, I/O failures or monitor process pauses behave the same way. Check each important result before using it; arbitrary unattended command files are not made safe by this example.

8.8. Recover from FP and assembly errors

In the following short transcript, the lines beginning with `***` are guest replies; do not type them. All other listings in this procedure are labelled as host-shell preparation, FP input or command-file contents.

A rejected FP command does not necessarily end the process or require a machine reset. In an ordinary FP process, a nonexistent program name in the supplied image produces:

Guest FP transcript: missing-program commands and replies

```
nosuchprog
***fp name nosuchprog
lookup nosuchprog
***lookup nosuchprog unknown
```

FP then accepts the next keyboard command. For example, `lookup slang` still shows the installed assembler. Check spelling, catalog scope and command context before retrying. Do not reset or reload the monitor just to recover from this name error: that would discard the running guest session.

Assembly errors require a separate check. The supplied assembler can leave a partial result entry even after reporting failure. A catalog entry marked `2.4` is **not** proof that its contents are a successfully assembled program. Always inspect the final SLANG report before running the result. If output was redirected to a listing, inspect that listing too.

The tutorial bundle provides a deliberately invalid source as `examples/slang_error.sl`. It contains an instruction referring to `a1` without declaring that identifier. To reproduce this example, copy it and the valid integer source from another host shell, starting in the emulator's working directory:

Host shell: prepare the failed and valid assembly sources

```
cp examples/slang_error.sl exchange/badinput
cp examples/slang_integer.sl exchange/slinput
```

Use unused temporary guest names and an ordinary process with a temporary disk claim. With console output selected (`o c`), enter in FP:

Guest FP: assemble and inspect the deliberately invalid source

```
badsrc=fromhost badinput
badprog=slang badsrc
lookup badprog
```

The tested assembler reports `a1 undeclared`, followed by `***slang sorry 1 1/4/1`. Nevertheless, `lookup badprog` finds a temporary one-segment entry with content/entry `2.4` and length 4. Do not run it. This behavior was checked with both supported monitor configurations; do not assume that a different historical version removes failed results automatically.

Remove only that failed temporary result, then confirm its absence:

Guest FP: remove the partial result and confirm its absence

```
clear temp badprog
lookup badprog
```

The lookup now reports `***lookup badprog unknown`. The original source `badsrc` is still available for inspection; removing the result does not repair the source. For this example, import the separate valid source and assemble it under a new name instead:

Guest FP: assemble and export the separate valid source

```
fixsrc=fromhost slinput
fixprog=slang fixsrc
recovered=tohost conv.bin fixprog
```

This produces `slang ok 1/12/1` and the same instruction bytes as the preceding successful example in host file `exchange/recovered`. Choose a host destination that does not already exist. FP has remained in the same process throughout; no Escape, Control-E, `new`, or machine reset is needed for these two errors.

After keeping any wanted source or output, clean up the remaining example files:

Guest FP: remove the recovery example entries

```
clear temp badsrc fixsrc fixprog
```

The same undeclared-identifier error does not abort a command file either. For a bounded example, copy `examples/assembly_error.fp` to host file `exchange/asmerrin`, and the invalid `slang_error.sl` source to `exchange/badinput`. The command file contains:

Command-file contents: assembly_error.fp

```
batchbad=slang batchsrc
lookup batchbad
clear temp batchbad
lookup slang
```

With console output selected (`o c`), an ordinary FP process and unused guest names, import and run it:

Guest FP: run and remove the assembly-error command file

```
batchsrc=fromhost badinput
batchcmd=fromhost asmerrin
i batchcmd
lookup batchbad
lookup slang
clear temp batchsrc batchcmd
```

The file reports `a1 undeclared` and `***slang sorry 1 1/4/1`, then still executes the remaining lookups and cleanup. Its first lookup shows `batchbad` with content/entry `2.4` and length 4. The file's explicit `clear` removes that partial result; this is not automatic assembler cleanup. The subsequent keyboard lookup reports `***lookup batchbad unknown`, and `lookup slang` still works in the same FP process. Do not replace the cleanup line with a call to `batchbad`: its executable marker does not make it safe to run.

This command-file case is checked with both monitor/controller variants. It demonstrates continuation after this one SLANG error, not safe automatic recovery from all compilation failures or automatic

conditional execution of later commands. In a real job, inspect the assembly report before running or exporting a program as a successful result.

These checks cover a missing FP program name and one undeclared SLANG identifier, not recovery from every compiler, editor, process or catalog error. In particular, an unfinished interactive program may still be waiting for its own input rather than an FP command. Use the command-context and process-control instructions before deleting files or removing a process.

8.9. Demonstrations and diagnostic programs

Entry	What is known	Evidence
`geo`	Interactive geography program. Enter `geo`, answer `e` at `dansk/English ?`, and it reaches the English `ARE YOU READY?` prompt. The startup path is tested, not an entire game.	Tested startup
`geodata`	Associated geography data. The text contains numeric tables and country names such as Afghanistan, Albania, and Denmark. Preserve it with `geo`; it is data, not an FP command.	Documented from contents
`regnetest`	FP command file entered as `i regnetest`, not simply `regnetest`. It conditionally compiles `cpuregtest` as `aaa2`, runs that program with supplied simulation input, and repeats through `i regnetest`. One complete iteration has been checked, including both numerical checks and one successful/zero failed runs.	Contents inspected; tested opt-in integration run
`cpuregtest`	Algol source used by `regnetest`, not an unrelated executable diagnostic. Its header identifies MPROFILE, VER 8EU, 7-FEB-80, Kaare Lund Rasmussen; the program prints version 83.04.12. It models cooling and nickel diffusion in iron phases and includes numerical self-checks.	Source inspected; exercised through regnetest

Entry	What is known	Evidence
`montest`	Installed program with help for inspecting internal/external process and area descriptors, message buffers, chain tables and core/dumps. Help and limited own-process/disk descriptor fields are checked below; other fields and state-changing functions remain untested. Help also mentions segment locking and output-file operations: do not assume it is a harmless automatic monitor test.	Help and limited process/disk displays tested
`claimtest`	Installed binary with parameter and error messages. Its name suggests resource-claim diagnostics, but neither that purpose nor its parameter contract has been established from source or execution.	Binary inspected; operating guide unavailable
`giersim`	A GIER simulation program is installed; its binary includes GIER stop/error reports. No complete simulated GIER session has been verified.	Documented identity, untested operation

Do not treat diagnostics as harmless demonstrations merely because their names contain `test`. Use a disposable session or a working disk copy for unverified tools. The developer's automated `regnetest` checkpoint is opt-in; it is not rerun by every normal quick check.

Run one regnetest iteration

For the verified `regnetest` workflow, choose an unused process name `test` and start a user FP from the supervisor `s` with `new test size 100000 run`. In the resulting user FP, enter `i regnetest`. A complete iteration prints these results (spacing may vary):

Guest output: one completed regnetest iteration

```
sum af tal= 360.700643456 test ok
sum af tal= 1.85598980864'3 test ok
antal okruns = 1
antal fejlruns = 0
```

This iteration has also been checked through the normal `sim4000 -a` frontend with default IX enabled. One iteration also passes with `RC4000.protected.cfg` and its matching disk: `cpuregtest` is freshly compiled under protection before producing the same results. This extends the checked software subset, not every installed numerical program or an unlimited repeated run. No special protection bypass is required; keep writes disposable and use the normal interruption procedure below.

A completed iteration does not end the test session

Ctrl-E (or the configured emulator escape key) suspends execution and returns to the simulator prompt; it does not terminate the guest test process. Resuming the emulator can continue the test. Use a volatile disk overlay or a disposable working image for this diagnostic session; do not assume the four result lines mean that FP has returned to keyboard input.

The command file begins with `if 0.no`, followed by a grouped `aaa2=algol cpuregtest` and `mode 0.yes`. Compilation therefore depends on the FP mode-0 state; do not assume it rebuilds on every iteration. It then calls `aaa2`, supplies a title and thirteen numeric inputs, and loops back through `i regnetest`. The final `finis` is present after that recursive input block; do not treat it as a guaranteed one-iteration termination.

The two source checks compare accumulated simulation values with 360.7006434 and 1855.9898087, each with absolute tolerance below 0.0000001. Passing these checks is useful evidence for this workload, not exhaustive CPU or floating-point conformance. `cpuregtest` alone is not the documented FP invocation: use the command file so the compilation and input conventions match.

Tested montest calls

`montest` is an inspection tool rather than simply a pass/fail diagnostic. In a disposable session, choose an unused process name `mtest` and create a user FP from the supervisor `s` with `new mtest size 200000 run`. The following help calls have been checked with the supplied monitor:

Guest FP: request montest help

```
montest info
montest info internal
montest info external
montest info buf
montest info chain
montest info typein
montest info end
```

Each prints a `call:` explanation and returns to FP. These commands request help, not the actual descriptor, buffer or chain display. For example, `montest info internal` describes selectors such as `internal all` and `name.<name>`. The single-process and all-slot displays below are separately checked; help output alone is not evidence that a display works. The following short dialog has also been checked. Enter the lines one at a time; after the first line, `info chain` is a montest command, not an FP call:

Guest FP: enter the montest dialog

```
montest typein
```

montest command input: request help and leave the tool

```
info chain
end
```

Guest FP: confirm the return from montest

```
lookup montest
```

`info chain` displays help without reading a disk-chain descriptor. With the exact `montest typein` invocation above, `end` leaves the tool and returns to FP; the subsequent `lookup montest` confirms that FP is

handling commands again. Do not confuse `end` with the record-display directive `finis` described in the typein help. The following limited record dialog has also been checked:

Guest FP: enter the montest record example

```
montest typein
```

montest command input: select the process record

```
internal name.mtest
```

montest record prompt: repeat the record and leave record display

```
repeat
finis
```

montest command input: request help and return to FP

```
info chain
end
```

After `internal name.mtest` prints the descriptor, the record prompt is `>`. At that prompt, `repeat` displays the same process again; its descriptor address and allocation bounds remain the same. `finis` leaves record display and returns to montest's command input, not to FP. The following `info chain` confirms that montest is still active. Only `end` returns to FP for this invocation. Other navigation controls and `out <file>` have not been tested. Ctrl-E still suspends the emulator to its operator prompt; it does not perform the guest tool's `end` command.

For read-only inspection, the following calls return to FP:

Guest FP: inspect process, disk and catalog header

```
montest internal name.mtest
montest internal all
montest external name.disc
montest external devno.15
montest chain docname.disc
```

The first call shows your process's name and allocation; addresses vary by session. The all-slot display is long and includes unused descriptor slots: use `list` in `s` for an ordinary job list, and `claim` in FP for resource reporting. Both external selectors identify guest device 15, `disc`; the number is not a Telnet port or host filename. The chain call shows the auxiliary device-catalog header, not the separate main catalog.

Only selected identity, allocation and basic header fields have been checked with the supported monitor profiles. In particular, the external display's `main : 960 (!)` is a misleading label for the disk's encoded device number, not an address to follow or proof of corruption. Do not reuse printed core addresses or assume every displayed field is valid.

Descriptor displays are not a general monitor certification

The shipped monitor is modified, so descriptor layouts and assumptions in this program may not match the running system. Do not use its core/dump, locking or file-related functions on important work until a disposable-session test has established their behavior. Binary strings alone do not establish compatibility.

8.10. Startup, system files, and optional subsystems

These entries generally belong to system setup, not everyday FP program execution. The names below are guest catalog names; host `MONITOR`, `UNIT0`, and `RC4000.cfg` are separate host files.

Entry	Role and operator guidance	Evidence
`startarea`	Startup command/input file. Its initial text creates `startproc`, then feeds `startdate c` and `finis` to it. Changing it changes boot behavior; leave it intact for normal use.	Contents inspected; normal boot tested
`startdate`	Guest date-initialization program called by `startarea`. This is the source of the startup date prompt, not a text script or the later `s` command `date`.	Tested during boot
`catalog`	Binary monitor main catalog, distinct from device catalog `catdisc`. Inspect its 72 stored segments with `--list-main-catalog`; do not edit or treat it as text.	Tested stored-record listing and slot accounting
`stdmonitor`	Disk-resident standard monitor used by the optional DFC403/RC4819 setup. Export/generation and boot are tested; do not replace the default host monitor casually.	Tested optional workflow
`autoloader`, `manuloader`	Installed loader programs. Their individual calling conventions are not covered here.	Inventory only
`startsos`, `starttem`, `startprimo`	Text command files that create and start the corresponding subsystem processes. The MIPS/TS profile moves SOS's disk claim to disc1.	Startup checked in the MIPS/TS profile
`startmips`	Text command file that reads the three component scripts. Autostart reads the components directly to avoid excessive input nesting.	Contents inspected; see the MIPS/TS chapter
`bsos`, `btem`, `bprimo`	Installed subsystem binaries for the historical MIPS/TS environment. These are not ordinary stand-alone FP utilities.	Historical distribution documentation

Entry	Role and operator guidance	Evidence
`sostrim`, `temtrim`, `primotrim`	Subsystem configuration/generation material. For example, `primotrim` contains editor-driven generation options. Presence does not imply the corresponding source files are installed.	Contents inspected; generation untested
`upsoscat`	Historical SOS user-catalog creation tool, used to prepare the demonstration account in UNIT0.mipsts.	Fixed catalog and login checked
`startsysadm`	Command file requesting an administrative process with permanent disk resources and explicit base/login ranges. Not a recommended beginner startup path.	Contents inspected; untested

The default disk has no SOS user catalog (`soscat`) and its subsystem scripts are not a ready-to-run MIPS/TS setup. The separate `UNIT0.mipsts` and `UNIT1.mipsts` images provide an account catalog, adjusted disk claim and tested automatic startup. See MIPS/TS (p. 53) for the complete procedure. Do not run the default disk's scripts merely to inspect them.

A safe way to inspect a known text script in an ordinary FP process is to export it without executing it, for example `sosstart=tohost startsos`, then read host `exchange/sosstart`. This export has been checked byte for byte; it does not execute the script or test subsystem startup. Never edit or reimport such a file over the installed entry without a recoverable working disk copy.

8.11. Libraries and catalog entry points

`algollib` and `oldalgollib` support Algol programs. Other installed library areas include `textproclib`, `bjbandelib`, `getreci`, `readaline`, `opensq`, `resultcf`, `headfilei`, and `headm`. Some have specialized interfaces which are not described by this operator manual. Preserve them: deleting an apparently unused library can break a program loaded later.

Many additional names, such as `sin`, `atan`, `write`, and `readchar`, refer to procedures in these shared areas. They are not all callable FP commands. Likewise, entries describing input/output devices are not text files. The catalog's `area`/`non-area` distinction is an allocation distinction, not a reliable executable/text/library classification. In particular, `edit` is `non-area` but callable, while `geodata` is `area` but contains data.

The historical distribution describes an important Algol compatibility choice: the installed system defaults to older versions of certain monitor interface procedures. Newer variants have `n`-prefixed names such as `nmonitor` and `nsystem`. Do not rename these interfaces just to modernize their spelling; parameter conventions differ and existing binaries may depend on the old defaults.

8.12. Other installed tools: not yet an operating guide

The following names are useful leads when exploring the image, but their full calling conventions have not been verified. These groupings are orientation, partly inferred from names, not tested capability claims:

- Catalog/storage-related candidates: `slicelist`, `discsave`, `disccopy`, `delete`, `rebase`, `basemove`, `fastmove`, `moveto`, and `maintenance`.
- Inspection or command-related candidates: `shownews`, `bdisplay`, `com`, `trace`, `snus`, and `termspec`. In particular, do not assume that `com` is a host serial-port service or that `trace` controls the simulator's host-side trace interface.
- Additional shared utility entries: `entry`, `set`, `permanent`, `print`, `binin`, `binout`, `mode`, `repeat`, and `end`. The `i/o` workflow and simple `if ok` conditions are tested above; advanced batch-control interactions remain unverified.

Some of these tools can alter catalog scope, disk allocation, or system state. For normal work use the tested file commands above and the simulator's host-side disk-copy/persistence facilities. A future documented example must establish the actual interface before presenting an unverified tool as a safe operator procedure.

To enumerate the current image without booting or changing it, run `./sim4000 --list-catalog UNIT0` (device catalog) or `./sim4000 --list-main-catalog UNIT0` (stored main catalog) from the emulator's working directory. In a running FP process, `lookup name` inspects a specific entry in the current guest view. Neither listing alone proves that a program is usable, and unallocated historical text remaining in disk sectors is not an installed file.

9. Host file exchange

This chapter's recipes use the default compatibility profile. With `RC4000.protected.cfg`, use `rcin`/`rcout` in place of `fromhost`/`tohost`. Both profiles use `exchange/`. The original transfer binaries are not compatible with strict protection; see Choose a maintained-monitor profile (p. 70).

9.1. Choose the native disk image

The default `src/disks/UNIT0` image contains rebuilt Algol programs named `fromhost` and `tohost`. The unchanged historical image is retained as `src/disks/UNIT0.original`, containing `fromdos` and `todos`. Normal CMake builds copy both images, and `RC4000.cfg` selects the native one:

Host shell: start the native image from the build directory

```
cd build
./sim4000 -a
```

An ordinary build uses the shipped native image; no guest compilation or image rebuilding is required. Keep the supplied image unchanged and use a separate working copy or persistent overlay for guest file changes; see Configuration and disk persistence (p. 70).

In an FP user process on that image, the basic text-transfer form is:

Guest FP: import text and export it under a new host name

```
hfsource=fromhost hfinput
hfoutput=tohost hfsource
```

The native programs default to text mode 2 and accept `conv.text`, `conv.iso`, and `conv.bin` (also `conv.no`). Their diagnostics use host terminology. They still address the historical monitor process `dos`; leave that guest endpoint name unchanged. The old command names no longer exist in `UNIT0`. If you separately have `UNIT0.original`, explicitly select it with `--disk UNIT0.original` to run the original programs instead.

9.2. Configure the exchange directory

The shipped configuration enables the following declaration. CMake creates `exchange` beside the runtime `RC4000.cfg`; create that directory yourself when arranging runtime files outside a CMake build tree:

RC4000.cfg contents: enable host file exchange

```
DEFINE HOSTFILES DEVNO=63 INTNO=21 ROOT=exchange
```

A relative `ROOT` is resolved against the selected configuration file's directory, not against the guest catalog. The root must already exist. Guest filenames are case-sensitive relative Unix paths beneath it; existing subdirectories are supported. Absolute paths, `./..` components, backslashes, drive prefixes, and symlinks are rejected. Exports create new regular files with mode 0600 and never replace existing files. The service is disabled unless declared. `show devices` reports the configured root and whether a guest file is open.

9.3. Import and export text

For a tested text workflow, put an ASCII file named `hfinput` in `exchange`, boot the monitor, and create or select an FP user process with a temporary disk resource claim as described in the file-work examples (p. 15). Choose an unused guest name `hfsource` and ensure that the host destination `exchange/hfoutput` does not exist. In that process:

Guest FP: run the text-transfer example

```
hfsource=fromhost hfinput
hfoutput=tohost hfsource
```

In `hfsource=fromhost hfinput`, `hfsource` names the new guest catalog entry and `hfinput` names a host file beneath the exchange root. In `hfoutput=tohost hfsource`, the left-hand name instead selects the host export destination; it does not create another guest catalog entry. No host shell is involved in either FP call.

The exported file is `exchange/hfoutput`. Each utility reports a byte count when it finishes. Those counts describe RC transfers and may include padding or catalog allocation; they are not necessarily the host file size.

9.4. Choose text or binary transfer

For a checked text-comparison workflow using protected exports and ordinary host `cmp`/`diff`, see Compare protected guest text on the host (p. 34). Those host tools are distinct from the incompatible protected guest `cmp`.

The native default text mode 2 passes character bytes through without a PC code-page translation. Imports skip NUL and CR and append the RC end marker (25); exports skip NUL and CR and stop at that marker. Host text exchanged in this mode uses LF. ASCII source files are supported; UTF-8 and Danish character conversion are not implemented yet. In the rebuilt native utilities, `conv.text` and `conv.iso` select the same mode 2; `iso` does not provide an additional encoding conversion. `conv.bin` mode 0 preserves all transferred bytes, including control characters and high bytes; RC files may still contain word/segment padding. For a separate binary example, use fresh guest and host names (or keep this example in a separate session with a different host destination):

Guest FP: transfer binary data without text filtering

```
hfsource=fromhost conv.bin hfinput
hfoutput=tohost conv.bin hfsource
```

Text transfer is not a byte-for-byte archive

Text mode filters control bytes and uses an end marker. Use `conv.bin` (equivalently `conv.no`) on both import and export when those bytes matter. Binary export preserves transferred data, but it does not remove guest allocation padding or recover an original host length automatically. Do not apply the text example to an executable or a binary backup.

9.5. Protect host files and interpret failures

Before retrying a failed transfer, check the utility's final diagnostic, the selected FP process, available temporary resources, the input filename and the exchange-root configuration. An existing export destination is deliberately refused: choose another host name rather than deleting a wanted file. Path components must name existing directories; the service does not create a directory tree. `show devices` is an emulator command, not an FP command; suspend execution before using it.

Host exports have their own lifetime: ending FP, removing a guest catalog entry or discarding a guest disk overlay does not remove a file already exported to the host. Keep wanted exports separately and inspect an interrupted or failed export before treating it as a complete result. The 0600 creation mode limits access to the host owner; it is intentional, not a sign of a failed transfer.

Inside the supplied monitor the historical endpoint remains process `dos` on device 63. The host service is called `HOSTFILES`; that guest process name remains necessary for compatibility with that monitor. Place files intentionally in the exchange root: it is a filesystem boundary, not a separate permission domain for files or hard links created there by the host user.

10. MIPS/TS: a terminal session

10.1. What this environment provides

MIPS/TS adds terminal and job services to the monitor. The supplied profile starts three resident subsystems: TEM manages terminal sessions, SOS handles account login and the user's process, and PRIMO provides peripheral transport services. SOS is the command interface used to log in; TEM is entered by the `run` command. Once enrolled, the user works in FP, using the guest commands described elsewhere in this manual. SOS is not a replacement for either FP or the emulator's `sim4000>` operator interface.

Historically, SOS means *Swopping Online System.* A SOS job is a process created from a named description in its user catalog. In this demonstration, `test` names that description: it supplies the initial program, memory limits, password and disk claims. It is not a host account, nor does logging in create a Unix-like home directory. The FP command `scope user` describes guest catalog retention, not a host user's ownership or a security boundary.

This is a ready-to-try demonstration with one configured account. Automatic startup, TEM-mediated login, incorrect-password rejection, FP commands and normal logout have been checked. It uses the compatibility monitor with IX enabled and storage protection off. Keep this unprotected profile for MIPS/TS operation. The standalone protected-monitor profile is separate; do not enable protection in this configuration or substitute its disk images.

Local demonstration credentials only

The account and password are both `test`. Telnet has no encryption; the supplied listener binds only to `127.0.0.1`. Do not expose this profile to a network or treat it as a secure multiuser service.

Choose a checked workflow

For a first visit, start the supplied profile, log in, try `message`/`lookup` and finish normally. The later exercises are independent: printing does not require the retained-file or batch exercise. Use one `test` job at a time, and finish it before submitting batch work.

Task	Main command context	Procedure
First terminal session	Host shell, then remote SOS/FP	Start (p. 54), then log in (p. 54)
Keep a small guest file	Preparation FP, then remote FP	Retained-file exercise (p. 59)
Run a small jobfile	Preparation FP, then local SOS	Batch exercise (p. 57)
Print guest text	Local PRIMO and administrative FP	Sequential printing (p. 64)
Resume a lost connection	Remote FP; no new login	Reconnect (p. 62), including known partial input (p. 62)
Check a job or service	Local guest system `s`	Process status (p. 63)
Unexpected failure	Local operator; preserve the job	Preserve evidence and work (p. 67)
Stop or back up	Remote FP, then local operator	Normal stop and backup (p. 61)

Expected diagnostics are stated beside the relevant commands. In particular, an intentional `lookup ... unknown` is different from a failed import, an abnormal SOS removal or an unfinished PRIMO transport. Do not continue past an unexpected error just because another prompt or bell appears.

10.2. Select the files and start

The ordinary build directory and binary archive contain the prepared files:

File	Purpose
`sim4000`	Emulator executable
`MONITOR`	Compatibility monitor, shared with the default profile
`UNIT0.mipsts`	System disk, account catalog and automatic startup
`UNIT1.mipsts`	Prepared second disk, mounted as `disc1`
`RC4000.mipsts.cfg`	Matching devices, Telnet and persistence settings

Keep these files together and create `exchange/` for host transfers. No Python, preparation utility or guest rebuilding is needed to run them. Do not substitute the default or protected disk for either MIPS/TS disk.

In a host shell, change to the build or unpacked distribution directory:

Host shell: start the MIPS/TS profile

```
./sim4000 -a -c RC4000.mipsts.cfg
```

The disk's startup file mounts the second disk, sets a fixed test date of 30 September 1986, and starts TEM, PRIMO and SOS without a date prompt. Wait for `sos started`; the release messages can arrive in a different order. A banner alone is not a login prompt. Leave this terminal running; it remains the local monitor/operator console.

10.3. Connect and log in

Open a second host terminal and connect to the configured port:

Second host terminal: connect

```
telnet 127.0.0.1 4004
```

Press Ctrl-A to request guest attention. After `att`, type `sos` and press Enter. At the SOS prompt enter the complete line:

SOS: start a session through TEM

```
run test pass test
```

A successful enrollment reports `from tem` and `test enrolled`, followed by the FP input prompt (a bell rather than a printed shell-style prompt). The inline password syntax above is the checked procedure; do not rely on a separate interactive password dialogue.

Remote FP: a first session

```
message hello
lookup fp
finis
```

The first command prints a message; the second identifies the FP program on the system disk. `finis` ends the user process, and SOS reports `test removed after finis`. To log in again, request attention with Ctrl-A, select `sos`, and repeat the login command. Only one terminal account is provisioned; adding a host listener alone does not configure an additional SOS terminal or account.

Recognize the command context

The same terminal carries commands for different interpreters. Attention selects a recipient; it does not log out, remove a job or suspend the whole emulator. Enter one command at a time and wait for its complete reply.

Context	How to recognize or select it	Commands in this chapter
Host shell	Before starting the emulator, or a separate host terminal	`./sim4000`, `telnet`
Emulator operator	Ctrl-E in the original terminal; `sim4000>`	`continue`, `quit`
Guest system `s`	Escape in the original terminal, then `s`	`list` for process inspection
SOS	Ctrl-A in Telnet, then `sos`; selection ends at `>` and a bell	`run test pass test` or `go test pass test`
FP in job `test`	Successful enrollment, then the FP input bell	`message`, `lookup`, `finis`

Do not type `run test pass test` at `sim4000>`: the emulator's `run` command controls CPU execution, not a SOS login. Similarly, `finis` belongs to the user FP, whereas `quit` exits the host emulator. TEM's `from tem` header identifies the terminal path; it does not mean that ordinary FP commands must be entered at a TEM setup prompt. Ctrl-E in the original terminal suspends the entire emulated machine, including remote jobs. Enter `continue` at `sim4000>` before expecting more Telnet output. Guest attention alone does not suspend CPU execution.

Direct and TEM-mediated sessions

The recommended login uses `run`, routing terminal traffic through SOS and TEM. TEM supplies terminal-output spooling. A direct alternative, also checked with this demonstration account, is:

SOS: alternative direct session, after finishing the previous job

```
go test pass test
```

`go` uses SOS without TEM terminal spooling. It still creates the same kind of FP job with the account's resource limits; it does not switch monitor profiles or disable a device. Expect `test enrolled` but no `from tem` header in the direct-login reply. Use one form or the other, not both for simultaneous logins to this single-account setup.

10.4. Small jobs and resource limits

The first `message`/`lookup` exercise is intentionally small. The login starts FP, not a complete modern shell environment; guest program names still refer to installed RC catalog entries. See Files, catalog entries, and Algol programs (p. 15) for the general FP notation, but do not copy its process creation or resource assumptions into a SOS session.

The demonstration description permits roughly 20,000–30,000 RC halfwords of job memory. Small FP commands and fixed batch allocation/copy examples have been checked; the larger Algol tutorial uses a different process with larger claims. A resource error here is not by itself an emulator defect.

Small input does not imply a small allocation

The supplied `fromhost` can allocate 36 segments even for a short text. That exceeds this account's 30-segment temporary claim. Do not use the general host-import tutorial unchanged in the demo job, and do not increase free disk capacity or relabel `disc1` to address an account claim error. The account's allowance, not merely the length of the host text, limits allocation; device access is a separate requirement.

An enlarged account alone does not grant access to the host device. Without explicit device inclusion, a larger-account check receives `reserve host-file process result 2` from a SOS job. Treat that message as failure even though it has no leading asterisks; an empty guest file may already have been created. Use the administrative staging procedure for the supplied demo account.

Beyond the supplied account

The supplied images contain only `test`; editing a host configuration does not create guest accounts or enlarge their claims. A separately provisioned larger account has been checked for Algol compilation and retained program execution across restart. It is not an additional ready-made profile.

For that larger account, direct transfers have been checked after the operator enters `include 63` at the *SOS* command prompt, after job enrollment. Expect a timestamped `sos: ready`, not an FP bell. This grants host-device access to *all currently enrolled jobs*; it does not enlarge disk claims, configure future logins, or provide separate host directories per account. Only one job can reserve the device at a time. Repeat the deliberate inclusion after enrollment following a restart. Do not add this command to the demo's normal startup or treat it as permission to use the host directory indiscriminately. Two separately provisioned accounts also pass simultaneous sessions on devices 4 and 5, with separate standard/user bases and two SOS child-process slots. An extra Telnet listener alone does not provide this setup: equal standard bases cause `user conflict`, and insufficient child-process claims prevent the second job's creation. Normal logout and relogin are checked for that two-account setup. The idle reconnect procedure (p. 62) below instead covers the supplied single-account environment. Do not add an account or replace `soscat` in a working installation merely to follow a recipe for the supplied `test` account.

The corresponding checked batch fixtures import text in an administrative preparation process, copy it to a compact area, then make the compact file visible to the job. This is not an automatic feature of a remote import. Creating more accounts or changing `soscat` requires a deliberate guest-administration workflow; editing `RC4000.mipsts.cfg` alone does not change SOS memory or disk claims.

Background jobs and jobfiles

SOS also offers background execution using `batch`. A jobfile is an existing retained guest text file containing the FP commands to execute; it is not a host shell script or a filename beneath `exchange/`. The following small exercise has been checked on the supplied unprotected profile, including both password forms and cleanup. Finish any interactive `test` session first: this recipe does not demonstrate concurrent jobs.

Prepare a compact jobfile

Use private runtime copies and their overlays. The supplied image has no `batchjob`; do not overwrite an entry left by another exercise. Create a new host file without replacing an existing one:

Host shell: create the batch input

```
(set -C; printf 'message batchprobe\nfinis\n' > exchange/batchinput)
```

Its two lines are FP commands, not SOS commands:

Host file contents: exchange/batchinput

```
message batchprobe
finis
```

In the original terminal press Escape, select `s` and allocate an administrative preparation FP with intervals matching the demo account:

Local Guest-s: allocate batch preparation

```
all bprep base 0 0 login 0 0 user 0 0 project 0 0 run
```

Wait for `to bprep` and the FP bell, then check the name:

Local FP bprep: check the target is absent

```
lookup batchjob
```

Expect `***lookup batchjob unknown`. If the name is found, stop and preserve that file. Otherwise prepare the compact retained guest input:

Local FP bprep: import and retain the jobfile

```
batchtmp=fromhost batchinput
batchjob=copy batchtmp
scope user batchjob
lookup batchjob
finis
```

The descriptor should say `=set 1 disc` and `; user`. Wait for `pause bprep finis fp`. The import uses administrative resources and `copy` compacts it; the batch job must not try to import its own input with the demo account's smaller temporary claim. Retention keeps the jobfile available after the preparation FP finishes. The 0/0 intervals establish visibility; a host pathname by itself does not.

Submit and recognize normal completion

Still in the original terminal, press Escape, select `sos` and wait for `>` and its bell. Enter one line:

Local SOS: submit the prepared jobfile

```
batch test jobfile batchjob
```

SOS asks `>password` and rings the bell. Only at that question enter:

SOS password question: answer for the demo account

```
test
```

The dummy password can be visible; do not infer masked input or secure authentication. The expected result, with a variable timestamp, is:

Expected local output: completed batch job

```
08.00 sos: test enrolled
 batchprobe
08.00 sos: test removed after finis
```

Enrollment alone is not completion. Wait for the actual message and removal *after finis*; `removed after break` is a failure, not the expected result. The jobfile's `finis` terminates the batch FP automatically: do not type FP commands at SOS while waiting. Select `s` and use `list` if needed; the services remain present and `test` is gone.

After normal removal, the jobfile is unchanged and can be submitted again. Request fresh attention and select SOS for the independently checked inline alternative:

Local SOS: repeat with the inline dummy password

```
batch test jobfile batchjob pass test
```

This form does not ask a second password question. Expect the same single enrollment, message and normal removal. Never resubmit merely because a reply is delayed: first establish whether the previous job still exists.

Check and remove the exercise input

Select `s` and create the same `bprep` FP again. Export the existing guest file to a new host filename, then deliberately delete the exercise entry:

Cleanup removes the retained jobfile

Use these commands only for the `batchjob` created above and only after both jobs have finished. They do not cancel a running job or remove host input and output files. Preserve a backup instead if the guest file is wanted.

Local FP bprep: export and remove the exercise jobfile

```
batchcheck=tohost batchjob
clear user batchjob
lookup batchjob
finis
```

Expect `***lookup batchjob unknown` and `pause bprep finis fp`. The host export must have succeeded before deletion; it refuses an existing `exchange/batchcheck`. Compare the independent host copies with `cmp exchange/batchinput exchange/batchcheck`. No output and status zero mean that execution preserved the jobfile bytes.

Historically, SOS schedules batch work in the background relative to interactive `go`/`run` jobs. A jobfile supplies input while the creating terminal receives output; this exercise submits from the local console. It does not establish arbitrary output destinations or remote batch submission. The plain `batch test` form does not supply that jobfile and is not the checked recipe for unattended work. Do not disconnect as a substitute for `finis`, or assume that this single-account demonstration establishes concurrent-job operation.

10.5. Files, persistence and shutdown

The demonstration account has a deliberately small disk claim: 30 temporary segments/six entries at key 0, and retained-file claims of two entries/six segments at each of keys 1 through 3. It is suitable for small FP experiments, not large compilation or data workloads. Retained files still need `scope user`; temporary files may be removed when the process finishes. See Keep guest files across sessions (p. 18). Do not assume an administrative FP's broader resource claims apply to this account. The second disk primarily supplies SOS's permanent working space; its existence does not automatically give the account that free space.

Unlike the default profile, this configuration retains writes automatically in `UNIT0.mipsts.overlay` and `UNIT1.mipsts.overlay`. Both base images remain read-only. Keep the two sidecars with their exact base images and configuration. A complete restart with automatic subsystem startup and exact readback of administrative and remote-created user-scope files has been checked; this does not certify every application or file-recovery path.

Keep a small file in the demo account

This exercise uses the supplied unprotected profile, one remote `test` session at a time and the existing small account quota. Work on private copies with their own overlays, not historical or shipped base images in write-through mode. The names `keepseed` and `keepsake` must be unused; do not overwrite files from earlier work. Start the profile as above.

Prepare the compact input

First prepare a short host text. This subshell refuses to overwrite an existing `exchange/keepinput`:

Host shell: create the exercise input

```
(set -C; printf 'MIPS/TS file retained across sessions.\n' > exchange/keepinput)
```

The file contains one line and a final LF:

Host file contents: exchange/keepinput

```
MIPS/TS file retained across sessions.
```

In the original terminal, press Escape and select `s`. Create a separate administrative FP with catalog intervals matching the demo account:

Local Guest-s: create an administrative preparation FP

```
all verify base 0 0 login 0 0 user 0 0 project 0 0 run
```

Wait for `to verify` and the FP bell, then check both names:

Local FP verify: check that the exercise names are unused

```
lookup keepseed keepsake
```

Expect `***lookup keepseed unknown` and `***lookup keepsake unknown`. These diagnostics are intentional. If either name is found, stop and preserve that file. Otherwise create a compact, retained seed:

Local FP verify: import and compact the seed

```
keeptmp=fromhost keepinput
keepseed=copy keeptmp
scope user keepseed
finis
```

Wait for `pause verify finis fp`. The large import belongs only to this administrative process; `copy` makes the small seed fit in one segment. The explicit 0/0 intervals make it visible to the demo job. This does not enlarge the account's claims or create its own file on its behalf.

Create the account's retained copy

Connect by Telnet and log in with `run test pass test` as in Connect and log in (p. 54). At the remote FP bell:

Remote FP test: create and retain your own copy

```
keepsake=copy keepseed
scope user keepsake
lookup keepsake
c=copy keepsake
finis
```

Expect a `keepsake =set 1 disc` descriptor ending in `; user`, the exact input line on the terminal and normal SOS removal after `finis`. Plain `copy keepsake` reports copy statistics; `c=copy keepsake` selects console output and displays the text. The remote job creates and retains `keepsake`; the seed is only its input.

Read it after login and restart

Request attention, select SOS and log in again. Read the existing file without recreating it:

Remote FP test: verify the retained file in a later session

```
lookup keepsake
c=copy keepsake
finis
```

The descriptor must still say `; user` and the line must be unchanged.

For byte-exact verification, select `s` locally and create the same `verify` FP with 0/0 intervals. Before restarting use `keepcheck1`; after it use the new host export name `keepcheck2`:

Local FP verify: export the existing retained file

```
lookup keepsake
keepcheck1=tohost keepsake
finis
```

Both exports must be new host filenames; export refuses collisions. In a host shell compare each with `exchange/keepinput`, for example `cmp exchange/keepinput exchange/keepcheck1`. No output and exit status zero mean equal bytes. A base-image listing does not include overlays and is not a substitute for this live guest lookup/export.

After the first export, perform a complete normal stop and restart with the same two overlays, following Stop, restart and back up (p. 61). Log in again, repeat the read-only remote sequence, log out and export locally as `keepcheck2`. Do not repeat seed preparation or `keepsake=copy keepseed`: that would test recreation, not persistence. Compare the second export with the host input before proceeding to cleanup.

Remove only the exercise files

Deliberate cleanup deletes the exercise files

Only after checking the retained data, log in again and delete the remote copy below. This removes that guest file; it is not a logout or overlay-reset command. Keep a backup instead if its contents are wanted.

Remote FP test: remove only the exercise copy

```
clear user keepsake
lookup keepsake
finis
```

Expect `***lookup keepsake unknown`, followed by normal logout. Finally create the local `verify` FP again, enter `clear user keepseed`, check that both names are unknown and finish with `finis`. Host input/ export files remain in `exchange/`; deleting a guest area does not delete those independent host copies.

Stop, restart and back up

Before stopping, enter `finis` in the remote FP and wait for normal SOS removal. Disconnect the Telnet client. In the original emulator terminal, press Ctrl-E and enter `quit` at `sim4000>`. This exits the emulator; it is not a guest system-wide shutdown command. Restart with the same configuration and overlays to resume stored disk changes. The boot date is set again on each start; it is not the current host date.

To return to the prepared demonstration, stop the emulator and remove both MIPS/TS overlay files together. This loses all guest changes since the images were prepared. Never remove an active overlay, reuse a different profile's overlay, or run two emulators on the same pair. Do not run `kitlabel` on the supplied second disk: it is already initialized.

For a backup, stop the emulator first and copy the matching `UNIT0.mipsts`, `UNIT1.mipsts`, both overlay files, `RC4000.mipsts.cfg` and `MONITOR` as one set. Preserve wanted host files in `exchange/` separately.

An offline catalog listing of `UNIT0.mipsts` reads only that base image, not a running session's sidecars; use guest `lookup` when checking current files.

Reconnect after losing an idle FP connection

This procedure is checked only for the supplied unprotected profile when FP had finished its previous command and was waiting for input. Closing Telnet without `finis` leaves that guest job running; it is not a logout. Two successive disconnect/reconnect cycles preserve both a temporary file and a retained user file. A TCP close is not a guest terminal hard error.

Keep the same emulator instance running. In its local terminal you can inspect `test` with `list` in guest `s`; see status checks (p. 63). At the simulator prompt, `show telnet` should report device 4 as `listening` once the old connection is closed. If you suspended the CPU with Ctrl-E, enter `continue` before expecting remote guest replies.

Reconnect from a host terminal:

Host shell: reconnect to the same running instance

```
telnet 127.0.0.1 4004
```

Do not request attention or start another SOS job first. At the reconnected terminal, send this directly to the already waiting FP:

Remote FP: confirm the existing session is responsive

```
message resumed
```

Expect the message `resumed` and the FP bell. No new enrollment or password dialogue is expected. Use `lookup` on your known guest files before continuing work; temporary files still belong to this same job. Finish later with `finis` in the usual way. If the old job has already ended or no FP reply arrives, stop and inspect its status rather than replaying commands or removing the process blindly.

Reconnection is not a new login or a saved-session restore

Anyone who can reach this local endpoint may resume the existing session without its password. Keep the listener on loopback and restrict access to trusted local users. Do not expose it as an authenticated remote service.
Lost output is not replayed. This clean-prompt recipe does not cover unknown pending input, active transfers, programs still running, terminal hard errors or emulator restart/crash recovery. Preserve wanted files and complete commands normally before a planned disconnect; prefer `finis` when you intend to log out.

A known partial input line

Reconnecting does not clear characters already accepted by FP. The following two harmless cases have been checked on the supplied profile. They require that you know exactly what was typed and saw it echoed before the connection was lost. They are not a method for guessing the contents of an interrupted input buffer.

At a waiting *remote FP*, type the following prefix *without pressing Enter*, then disconnect Telnet without `finis`:

Remote FP: partial line, no Enter

```
message par
```

Reconnect to the same running emulator as above. Do not request attention or log in again. Enter only the missing suffix and submit it:

Reconnected remote FP: complete the existing line

```
tial
```

Expect `partial` followed by the FP bell: FP has executed `message partial`, not a new command named `tial`.

For a correction example, start another line, again without Enter:

Remote FP: known partial line to correct, no Enter

```
message wrong
```

After disconnecting and reconnecting, press Ctrl-H exactly five times to erase the known suffix `wrong`, then type `right` and Enter. Ctrl-H is ASCII Backspace; the physical Backspace key is equivalent only when its DEL output is mapped to Ctrl-H by the active configuration. Expect `right` and the FP bell. This does not require removing the job, discarding files or restarting the emulator.

Do not press Enter on an unknown partial line, and do not send a fresh whole command on top of it. Bytes typed just before a network failure may not have reached the guest; the examples only establish recovery of a fully echoed known prefix. For anything uncertain, use the non-destructive diagnostic steps (p. 67).

10.6. Check status without disturbing the job

Keep the Telnet session for the user job. To inspect the guest from its original local terminal, press Escape, select `s`, then enter:

Local guest system process s: inspect processes

```
list
```

The list should include `tem`, `primo` and `sos`. During a login it also includes `test` under parent `sos`; after normal `finis`, that child disappears. A waiting interactive child can be displayed as `stop` with stop count zero. That single state word is not proof of a crash: successful FP input/output and normal logout are the relevant checks. The resident services should remain active and have zero stop counts. Process addresses and exact scheduling states can change; do not copy addresses from an example into commands.

`list` here is an `s` command. Do not replace it with SOS `display`: that input has returned `command unknown` in the checked setup. Nor does a SOS `ready` response to a diagnostic `call` prove login, password validation or a terminal connection. Use the actual enrollment and FP exercise instead. Do not use `proc test remove`, SOS `kill`, or a reset as routine logout commands; they can discard the job's temporary work.

10.7. Print through PRIMO

The supplied configuration opens `printer.mipsts.txt` automatically. No change to the disk images, `startprimo` or PRIMO generation is required. This recipe uses the original local terminal for administration, not the resource-limited `test` login. Start the MIPS/TS profile and wait for `sos started` before proceeding.

In a second host terminal, prepare the two input files. Do not overwrite existing exchange files without checking their contents first:

Host shell: prepare a program and printable text

```
cp examples/primo_print.alg exchange/prbsource
printf 'Hello from PRIMO.\n' > exchange/prbinput
```

In the original terminal, press Escape, enter `primo`, and wait for its = prompt and bell. Sign up the real printer (kind 14):

Guest PRIMO operator: assign the physical printer

```
signup printer 14
```

Wait for the timestamped `primo:ready` reply. To inspect the assignment, request attention again with Escape, select `primo`, then enter `display`. Expect a `printer` row ending in `,idle`. Each operator command requires a fresh attention/selection; the ready reply is not an invitation to type at a persistent prompt.

Press Escape and select `s`, then allocate an administrative FP:

Guest s: allocate a temporary printing session

```
all prbprep size 150000 temp disc 500 20 run
```

Wait for `to prbprep` and the FP bell. The following creates only temporary guest entries in this process; do not substitute existing retained files as output targets:

Guest FP: import, compile and submit one transport

```
psrc=fromhost prbsource
ptmp=fromhost prbinput
pin=copy ptmp
plp=set lp printer
prbprog=algol psrc connect.no
prbprog
```

Compilation must end with `algol end`, without errors. The program calls the installed `transfer` procedure to define a transport, then waits for it. Expect the following, with a positive, variable transport number:

Expected FP result: accepted and completed transport

```
primo define 0 reply 0 transport 82
primo wait 0 reply 0 state 5 cause -1
end         26
```

The numeric `end` value is also variable. State 5 with cause -1 denotes normal completion in this checked recipe. At that point the host file already contains `Hello from PRIMO.` followed by LF; no emulator

shutdown is needed to flush it. The payload must not merely appear on the console. `plp=set lp console1` would select a console substitute and is deliberately not used here.

To print another text, replace the host `exchange/prbinput` with the new input, then repeat `ptmp=fromhost prbinput`, `pin=copy ptmp` and `prbprog` in the same FP. Do not change or remove `pin` before the previous transport has completed. Keep lines within 132 columns; the printer chapter describes character/control mapping and buffering limits. The current PRIMO trimming emits the input without job banners or extra page separators; do not assume that other PRIMO installations behave identically.

After completion, enter `finis` in FP and wait for `pause prbprep finis fp`. Select PRIMO again and enter `signoff printer`; wait for ready. A fresh `display` must no longer show the assignment. Signoff removes the operator assignment; it does not detach the emulator output file. Keep completed output when restarting: the next run appends to it. The checked workflow includes repeated short and 160-line jobs, signoff, two boots with persistent disk overlays, and successful TEM/SOS login/logout after printing on each boot.

This is a bounded administrative printing example, not a general SOS user print command, concurrent queue test or protected MIPS/TS acceptance. If an operation reports an error or waits for an operator, retain its diagnostics and do not report the transport as completed.

Recover a job submitted with output detached

When a transport is submitted while the emulator output is detached, the checked PRIMO version reports `primo:***printer end document`. This is an operator request, not successful completion. Select PRIMO and enter `display`: the `printer` row identifies the input area `pin` and ends in `,waiting`. Leave the submitting FP and its input intact. Do not run `prbprog` again to retry the same transport.

Suspend with Ctrl-E. Reconnect the existing, healthy output file in append mode, then resume CPU execution:

Emulator CLI: reconnect detached output

```
attach 7 printer.mipsts.txt append
continue
```

These paths are relative to the emulator's working directory. Select PRIMO with Escape, then `primo`, and issue:

Guest PRIMO operator: resume the waiting transport

```
start printer
```

Wait for both `primo:ready` and the submitting program's completion: `primo wait 0 reply 0 state 5 cause -1`, a normal FP `end` and its input bell. The text is then present exactly once in the host file; earlier output is preserved. Use `start`, not `restart`: the latter deliberately starts the transport again from the beginning and can repeat printed text. Do not use an exclusive attach to an existing file or delete the output to make attachment succeed.

The acceptance test deliberately detaches output between completed jobs, submits one new job, confirms the waiting state and unchanged file, then reattaches and resumes. It also checks healthy TEM/PRIMO/SOS while waiting, normal login/logout afterward and preservation across emulator restart. This does not certify recovery from partial host writes, disk-full errors, mid-job paper loss, unrestricted concurrent jobs or an emulator crash. Preserve diagnostics and output rather than applying this recipe blindly to another failure.

Multiple outstanding jobs: a bounded limit

Two transports from separate administrative processes have been checked: both are accepted while the printer is unavailable, then print exactly once in submission order after reattachment and `start printer`. A further transport completes on the same PRIMO instance after the second client is removed. This does not establish arbitrary queue capacity or every scheduling policy.

There is an important restriction: the second simultaneous blocking wait returns reply 3 (*missing resources*) in this session. The developer fixture handles that reply explicitly by querying transport state with action 4 until completion, within finite test limits. It does not resubmit the data or treat the refused wait as a successful print. The installed `primotrim` generation text sets `waitops := 1`, unlike `waittrans := 300`: queued transports and blocking waits have different limits. A private test with eleven rather than ten runtime message buffers still refuses the second wait. This strongly points to the generated PRIMO waiter limit, not an emulator printer setting. Increasing startup buffers or editing the generation text alone does not regenerate the installed program. The PRIMO binary and shipped startup remain unchanged.

The supplied `primo_print.alg` example is intentionally synchronous and has no polling fallback. Use the sequential recipe above for ordinary printing. If its wait reports a nonzero reply, preserve the transport number and diagnostics: the original job may still be queued. Submitting it again can create duplicate output. Limited test coverage is not a reason to enlarge claims blindly or run `primotrim` on the installed system.

10.8. Troubleshooting and current limits

First identify the context in the command context guide (p. 55). Preserve the complete diagnostic and the selected config before changing anything. The checks below do not authorize deleting wanted data or resetting a working profile.

Connection refused. Wait for startup, check that the emulator is still running and that port 4004 is free. Check the selected configuration, not the default `RC4000.cfg`, which has no active listener.

No SOS prompt. Press Ctrl-A in Telnet, then select `sos`. If the local terminal is at `sim4000>`, enter `continue` there: Ctrl-E suspends the whole machine, including remote work. A running FP normally signals readiness with a bell, not a printed SOS or shell prompt.

Bad password. Use `run test pass test` at SOS, not at FP or the emulator prompt. For the batch password question, answer `test` only after `>password`; the inline batch form does not ask that question. See batch submission (p. 58). Do not submit duplicate jobs while diagnosing a delayed reply.

Insufficient resources or an existing session. Use one test job and finish it normally. A tiny host import can still exceed the 30-segment temporary claim; use the compact-input procedure (p. 59), not `kitlabel` or a larger disk. For a lost connection to an idle FP, use the bounded reconnect procedure (p. 62); it does not cover interrupted commands or a failed guest job. Extra accounts and simultaneous terminals require deliberate guest provisioning, not just another listener in the configuration.

File unknown after login or restart. Check whether it was retained with `scope user`, whether the administrative FP used the specified 0/0 intervals, and whether both matching overlays were kept. A fresh overlay starts from the supplied image, not the previous session's retained files. Inspect with guest `lookup` before recreating or deleting anything.

Only copy statistics appear. Use `c=copy keepsake` to display the exercise text. Plain `copy keepsake` is not the display command used in the retained-file recipe.

Host export refuses a file. An existing export name must not be overwritten. Preserve that host file and select a new name; for other export failures, retain the diagnostic rather than assuming a name collision. Do not run guest cleanup until the needed export succeeds.

Batch enrolled but not finished. Enrollment is only acceptance. The checked jobfile must contain both `message batchprobe` and `finis`. Wait for the message and `test removed after finis`; inspect `list` from local `s` before deciding whether another submission is safe.

Guest break or abnormal removal. Record the originating process and numbers. `removed after break` is not normal completion. Check memory/ storage claims before larger commands. Do not delete overlays containing wanted work, force-remove processes or reset as an automatic recovery measure.

Printer waiting or nonzero wait reply. Preserve the transport number and input area. Do not resubmit: the original job may still be queued. Follow detached-output recovery (p. 65) only for that specific checked failure, not disk-full or partial-write errors.

Date prompt or missing disc1. Select the matching `UNIT0.mipsts` and `UNIT1.mipsts` pair. Older overlays can preserve previous startup edits. Back up the stopped profile before resetting it; never initialize the supplied second disk again.

Preserve work after an unexpected failure

When none of the specific checked recipes applies, do not try a sequence of SOS `stop`, `start`, `break` or `kill` commands. Use the following order instead:

1. Stop sending remote input. Keep the complete terminal diagnostic and the last command, noting whether Enter was sent, whether its output finished, and which interpreter was receiving input. Do not replay an import, batch job or print request merely because its acknowledgement was lost.
2. In the original terminal, suspend execution with Ctrl-E and inspect the existing instance. These *emulator CLI* commands do not remove a job:

   Local emulator CLI: inspect before changing state

   ```
   show cpu
   show telnet
   show media
   ```

   Record the selected configuration and matching base/overlay names as well. While suspended, the guest cannot answer FP or SOS commands. Enter `continue` before a deliberate guest status check or export.
3. If the guest remains responsive, inspect processes through local `s` as in the status procedure (p. 63). Inspect wanted files with `lookup`; export them under new host names only through a healthy FP with the appropriate intervals and resources. Successful export, not a prompt alone, is the evidence that a wanted file was saved.
4. Prefer normal `finis` and shutdown only when the relevant jobs are usable. If this is not possible, preserve the running instance and its diagnostics before deciding to abandon it. Exiting loses core, pending input and uncompleted work; an overlay is not a resumable machine snapshot.

5. After the emulator has stopped, preserve both overlay files with their exact base images and configuration before further experiments. Never copy active disks/sidecars as a consistent backup, reset by relabelling `disc1`, or discard the only copy of wanted guest data.

This is a preservation procedure, not a promise to recover every failed job. Interrupted transfers/output, actual terminal hard errors, disk-full or partial host writes, power loss and emulator crashes have no general checked recovery sequence here. A future investigation must not be confused with a supported instruction to mutate the current guest.

Punch/reader workflows and general printer error recovery are not certified. The subsystem binaries are historical programs; their generation sources have not been established as available on the image. Do not run `sostrim`, `temtrim` or `primotrim` as setup commands.

The default `UNIT0` does not start MIPS/TS. In `UNIT0.mipsts`, `startarea` reads the three component scripts directly; an extra `read startmips` would exceed the monitor's two-level input stack and report `not allowed`. `startsos` places its normal 2500-segment/200-entry claim on `disc1`, and `soscat` contains the demo account. These differences are confined to the MIPS/TS images.

10.9. Historical reference and coverage

The historical *Swopping Online System (SOS)*, RCSL 31-D662 (February 1982), explains job creation in section 2.2.1, printed pages 14–15; terminal access in section 2.3, pages 20–21; and user-catalog resources in section 5.3.1. The *MIPS/TS Operating Guide*, RCSL 31-D585, second edition (October 1979), supplies operator and subsystem background. These manuals describe historical configurations, not a promise that all their devices, accounts or commands are configured here. Scanned OCR text can misread command syntax; prefer this chapter's checked recipes.

The ready-to-run coverage is one compatibility-monitor terminal session, not multiterminal application attachment, invisible-password input, arbitrary guest generation, general peripheral recovery, or protected MIPS/TS. The historical SOS intervention commands `stop`, `start`, `break` and `kill` are not a certified recovery sequence for this demonstration. In particular, historical `break` behavior depends on system trimming and need not preserve a job's files.

Part III.

Emulator and device reference

11. Configuration and disk persistence

11.1. Select CPU and device capabilities

The examples in this chapter are independent choices, not one shell script. Configuration snippets replace the corresponding active declaration; do not add a second device with the same number. Restart the emulator to use a changed configuration. The normal startup options are summarized in the command-line reference (p. 75).

`CPU IX=YES` enables the narrow index-instruction extension needed by the supplied bare `catstat`; this is also the default if the line is omitted. `CPU IX=NO` restores the baseline unassigned-opcode-61 behavior. The setting is fixed for the session and retained by `reset`; it is not stored in a core image. Command-line `--ix`/`--no-ix` take precedence.

Storage protection defaults to off when the directive is omitted, equivalent to `PROTECTION MODE=OFF`. Select `PROTECTION MODE=RC4000` to enable storage key, instruction-fetch and privilege checks. This is a separate, session-fixed setting, not an RC8000 mode or an IX option. It requires a separately rebuilt monitor and compatible software bindings; simply enabling it with the supplied MONITOR and transfer utilities is not a supported ordinary-use configuration. Use the matching protected configuration and initially keep disk writes disposable.

`CORESIZE=minimum,maximum` counts RC bytes (12-bit halfwords), not host bytes. The default is 1,048,576 RC bytes, stored in about 2 MiB of host data memory plus 0.5 MiB for word keys. The configured minimum is raised to 16,384; the maximum is capped at 8,388,607, and a minimum above that cap is rejected. This retained runtime limit is smaller than the CPU's unsigned 24-bit address space. Keep the default for the supplied monitor; changing memory size is not needed for normal use.

Startup rejects the recognized but unimplemented `RC433XRAM` device type before loading the monitor or opening media. RC433XRAM is intentionally out of scope: use the default volatile RC433X copy-on-write mode for nonpersistent writes instead. `--check-config` checks configuration syntax only, not runtime device support. Physical `RC433` is not a recognized configuration type; `RC433X` is the implemented emulator-specific disk interface.

11.2. Choose a maintained-monitor profile

The build directory and binary distribution include the compatibility, protected and MIPS/TS profiles ready to run. Compatibility remains the default; select it explicitly with `./sim4000 -a -c RC4000.compat.cfg`. Select the protected alternative with `./sim4000 -a -c RC4000.protected.cfg`. Keep its matching `MONITOR.protected` and `UNIT0.protected` files together. All profiles use the shared `exchange/` directory, CLASSIC terminals and IX. In protected FP sessions use `rcin`/`rcout` instead of `fromhost`/`tohost`. Some installed programs use newer-machine instructions and remain incompatible with strict RC4000 protection. The supplied `PROFILES.txt` explains the profile-specific software and persistence boundaries; switching profiles does not require replacing files. The installed `cmp` also pauses in `tofrom1` under protection; use compatibility for guest text comparison or export with `rcout` and compare the resulting host files. See Compare text files inside the guest (p. 32).

Process lifecycle, nested delegation, denied cross-process writes/code entry, privileged-operation faults, selected Algol/SLANG work and independent local/ Telnet sessions have bounded protected-profile tests. This does not certify every editor, numerical workload or optional subsystem on the disk, and it is not a host-security sandbox. Ordinary data reads across storage keys remain allowed by the RC4000 architecture.

Compatibility and protection default to volatile disk writes. MIPS/TS uses two persistent overlays by default; see MIPS/TS (p. 53) for its automatic startup and login procedure. To retain protected disk writes, replace its active disk declaration with:

Protected configuration: replace the disk declaration

```
DEFINE RC433X DEVNO=15 FILE=UNIT0.protected OVERLAY=UNIT0.protected.overlay
```

Never exchange base disks or overlays between profiles. To recover the compatibility environment, select `RC4000.compat.cfg`, rather than replacing its files with protected ones.

11.3. Add a second RC433X disk

This procedure has been checked with a fresh compatibility-profile session. It does not apply to the DFC403 profile. Work in a private runtime directory and use an unused device number; the example uses device 16, which the supplied maintained monitor supports as `disc1`.

First create a new starter image with the integrated disk-creation command (p. 79):

Host shell, private runtime directory

```
./sim4000 --mkdisc UNIT1
```

Keep the existing device-15 declaration and add this line to your private configuration. The persistent sidecar retains all writes without changing the blank base file:

Private configuration: additional disk

```
DEFINE RC433X DEVNO=16 FILE=UNIT1 OVERLAY=UNIT1.overlay
```

Boot normally and initialize the date. At attention, select `s`, then enter `all big run`. This creates an administrative FP named `big`; it is not the same as creating an ordinary user FP with a broad base interval. In that FP enter:

Administrative FP: initialize a newly created disk only

```
kitlabel 16 disc1 catdisc1 slow 36 9 2030
kiton devno.16
show
finis
```

Labelling initializes the selected guest disk

Run `kitlabel` only for the new blank disk on device 16. Do not use it to mount an existing disk: it can destroy the disk's guest catalog and files. The host-side overwrite refusal in `mkdisc` does not protect a guest disk from a later `kitlabel` command.

The mount report names `disc1` on device 16. Initially `show` reports 2025 free slices, or 18225 segments, on that disk. Its usable capacity is therefore larger than the starter file; the persistent overlay grows as

sectors are written. Retained guest files must still have an appropriate scope and be allocated on `disc1`, not merely on the original `disc`.

On later boots, create an administrative FP and use only `kiton devno.16`; do not repeat `kitlabel`. Keep `UNIT1` and its matching `UNIT1.overlay` together. A 207-segment retained text file has been checked byte for byte, with unchanged catalog metadata, after a complete emulator restart and remount. Automatic mounting and use by MIPS/TS are not part of this procedure.

The historical startup command `kit 16` is not available in `s` after its initialization-only catalog code has been removed when the first internal process is created. The installed FP utility uses the explicit `devno.16` form shown above.

11.4. Select the optional historical disk interface

For the historical DFC403/RC4819 interface, use the supplied `STDMON` and `RC4000.dfc403.cfg`. Both are copied to the build directory by an ordinary build. Include them in your runtime directory if you want this optional profile; no monitor-generation script or Python installation is needed.

Host shell, runtime directory: start the standard monitor

```
./sim4000 -a -c RC4000.dfc403.cfg
```

The supplied standard monitor was exported from disk-resident `stdmonitor` and checked by booting DFC403/RC4819 and exercising text/binary host transfers. Its regeneration is a developer workflow, not a runtime prerequisite. The ordinary configuration still uses RC433X.

DFC403 provides separate data and head-positioning interrupts; RC4819 uses 406 cylinders, 20 heads, and 9 sectors per track. Drive declarations use `DFC` and `UNITNO` after their controller, rather than a separate `DEVNO`. RC4819 images use volatile copy-on-write only; `--disk` can override the first image, but persistence options, runtime attach/detach, and geometry editing are not yet available for this interface. Optional `RC4819RAM` drives have a fixed capacity of 1..406 cylinders. `show devices` and `show media` include controller/unit identities. This models the historical command protocol, not physical seek/rotation timing or overlapped transfers.

11.5. Select runtime files and override paths

The normal runner reads `RC4000.cfg`. The configuration selects `MONITOR` and `UNIT0` relative to the configuration file:

Host shell, build directory: bounded run with configured paths

```
./sim4000 --run --config RC4000.cfg \
  --steps 1000000 --set-disc-load-flag
```

The monitor and disk paths can be overridden independently:

Host shell, build directory: override monitor and first disk paths

```
./sim4000 --run --config RC4000.cfg \
  --monitor MONITOR --disk UNIT0 --steps 200000
```

11.6. Choose how RC433X disk writes survive

Host persistence and guest file lifetime are independent. Even on a writable persistent disk, a temporary guest entry may be removed when its FP process ends. Use the guest retention procedure (p. 18) for wanted files; disk persistence alone does not promote them.

Disk writes normally remain in the volatile in-memory overlay. To retain them across restarts without modifying the base image, select a sidecar explicitly:

Host shell, build directory: select persistent RC433X writes

```
./sim4000 -a --disk-overlay UNIT0.overlay
```

The equivalent configuration replaces the RC433X declaration with:

RC4000.cfg contents: replace the disk declaration for persistence

```
DEFINE RC433X DEVNO=15 FILE=UNIT0 OVERLAY=UNIT0.overlay
```

The base image is still opened read-only. The sidecar records its size and FNV-1a checksum, so it is rejected if used with another base image. Each sector record has its own checksum. Updates are written to a temporary file, flushed, and atomically renamed after every accepted RC disk-write operation. The emulator rejects truncated files, damaged records, duplicate sectors, and an overlay path that refers to the base image.

Keep an overlay with its matching base image

Stop the emulator before copying a persistent base/sidecar pair. Keep the selected configuration with the pair and preserve the original base unchanged; the sidecar is not a complete disk or a machine-session snapshot. To return to the pristine disk state, stop the emulator and preferably move the wanted sidecar aside rather than delete it. Remove only the selected `.overlay` file. Never replace `UNIT0` with the sidecar: it is a versioned sparse change file, not a complete RC433X image.

11.7. Use an explicit complete working copy

For a complete writable disk rather than an overlay, copy the stopped base image to a new name and explicitly select write-through mode. This works in a standalone runtime directory and does not need Python:

Host shell, runtime directory: create and select a working copy

```
mkdir -p work
cp -i UNIT0 work/UNIT0.test
./sim4000 -a --disk work/UNIT0.test --disk-write-through
```

Direct disk writes modify the working copy immediately

`--disk-write-through` requires an explicit `--disk` argument and conflicts with `--disk-overlay`. It may modify and extend that file immediately. Create a separate working copy before starting; see Prepare a writable working copy (p. 18). The supplied `UNIT0` must never be a write-through target. Do not copy a disk while the emulator is writing it, and do not run two emulators writing the same disk or sidecar. A working copy preserves the original image, not the temporary guest entries or the live process state. Keep an untouched backup before trying unfamiliar guest commands. To abandon an experiment, stop the emulator, preserve any wanted host exports, and use a new copy of that backup. Do not replace a disk image beneath a running emulator, and do not reuse an overlay with a different base image.

For GNU-style startup syntax, aliases and mode restrictions, use the command-line reference (p. 75). Runtime RC433X `attach` choices, including direct-write `writable`, are described in disk attachment controls (p. 79).

11.8. Inspect stored catalog records and runtime limits

Diagnostic commands are also available as independent host-shell calls. They do not start the guest:

Host shell, build directory: independent offline checks

```
./sim4000 --check-config RC4000.cfg
./sim4000 --inspect-monitor MONITOR
./sim4000 --inspect-image UNIT0
./sim4000 --list-catalog UNIT0
./sim4000 --list-main-catalog UNIT0
```

`--list-catalog` follows the catalog and chain-table slice links rather than searching the image for printable strings. It prints the catalog identity and one row for every occupied entry. Positive sizes are reported as `area`; zero and negative sizes are reported conservatively as `non-area`. Tail fields whose higher-level meaning depends on the entry contents are retained as numeric `SHORTCL`, `FILE`, `BLOCK`, `CONT`, `ENTRY`, and `TAIL` columns. The command never writes the image and rejects malformed headers, out-of-range links, premature chains, and chain cycles. `--list-main-catalog` locates the positive-sized `catalog` area through the device catalog and follows its slice chain with the device's chain table. It requires the same document; arbitrary main-catalog names, other devices and auxiliary-catalog selection are not implemented. Its columns have the same format, but show stored main records, not a live namespace or a backing-file accessibility check.

The bounded `--run` runner remains available for reproducible tests. It attaches configured clock devices and basic consoles, sends the first console's output to standard output, reads its input from standard input, and advances deterministic device time once per 10,000 interpreter calls. On a terminal, input is character-at-a-time with local echo disabled; Escape acts as the historical operator key. Newline from a pipe is translated to RC Return. The original terminal settings are restored after a normal exit and after handled SIGINT or SIGTERM. Configured basic-console Telnet endpoints are serviced at the same deterministic intervals. Multiple local virtual-console screens remain deferred.

The runner resolves configured RC433X and overlay paths relative to the configuration file and opens the base image read-only. Without `OVERLAY`, disk writes are visible to later emulated reads through a sparse in-memory copy-on-write overlay and are discarded when the process exits.

The default and overlay modes do not modify the monitor or base disk image; explicit write-through mode does modify its selected working copy. For an end-to-end retained-file example, see Keep guest files across sessions (p. 18).

12. Starting and controlling the emulator

12.1. Select a startup mode

Shell options begin with `--` for long names or `-` for single-letter alternatives. They are different from commands typed at the `sim4000>` operator prompt and from guest FP commands. Use the full long-option names shown here.

`--ix` enables the optional opcode-61 index instruction; `--no-ix` disables it. They override `CPU IX=YES|NO` in the configuration for interactive and bounded runs. If both occur, the last option wins. The default is enabled for compatibility with the supplied software, not a claim about the historical hardware.

The operator examples below assume that the current directory is the CMake build directory. This preserves the default `RC4000.cfg` lookup and the flat runtime paths used by that configuration.

The interactive frontend is the normal operator entry point. `-a` is the convenient autostart form: it selects interactive mode, sets the monitor's disc load flag, and immediately runs the machine while using `RC4000.cfg` from the current directory. Control-E (or the configured emulator escape key) suspends that automatic run and opens the `sim4000>` prompt.

Host shell: alternative interactive startup forms

```
./sim4000 -a
./sim4000 -i -c RC4000.cfg
./sim4000 --interactive --config RC4000.cfg --set-disc-load-flag
```

The three startup lines are alternatives, not a sequence. Unlike `-a`, `-i` opens the operator prompt without immediately running. Adding `--set-disc-load-flag` requests guest disc startup when execution begins; it does not itself issue `run`. With no arguments, the program prints usage and exits. Select one execution mode; `--run` cannot be combined with `-i` or `-a`.

12.2. Override configuration and bound execution

The following startup options apply to interactive or bounded execution unless stated otherwise:

Option	Meaning and limits
`-c FILE`, `--config FILE`	Select configuration instead of `RC4000.cfg` in the current directory.
`--monitor FILE`	Override the monitor path selected by the configuration.
`--disk FILE`	Override the first configured disk image, not every disk device.
`--disk-overlay FILE`	Select a persistent overlay for the first disk; supported for RC433X, not RC4819.
`--disk-write-through`	Write directly to an explicitly selected RC433X `--disk FILE`; incompatible with `--disk-overlay`.
`--set-disc-load-flag`	Request the monitor's disc autostart path; distinct from the paper-tape `autoload` operator command.

Option	Meaning and limits
`--ix`, `--no-ix`	Override the configured optional opcode-61 capability; the last of these options wins.
`--run`	Execute without the interactive operator prompt, bounded by an instruction-call limit.
`--steps COUNT`	Set the bounded-run limit; default 1000000. This does not limit an interactive `run`.
`--trace FILE`	Write a deterministic trace during `--run`; not available with interactive startup.
`-h`, `--help`	Print startup usage and exit without starting the machine.

Configured paths are relative to the configuration file, while shell option path overrides are relative to the current directory. A command-line override takes precedence over its configuration setting; it does not rewrite the configuration file. See the runtime-file guide (p. 5) and disk persistence (p. 70) before moving runtime files or selecting a disk-writing mode.

Write-through changes the selected disk image

Use a disposable working copy with `--disk-write-through`, never the preserved original image. Without a persistent overlay or explicit write-through, ordinary disk writes use the default volatile overlay and disappear when the emulator exits. An instruction limit is not a guarantee that guest operations have reached a safe completion point.

For example, this shell invocation requests disc startup for a bounded run. It is not an interactive session and does not promise a complete monitor boot or completed guest job:

Host shell: bounded execution with an explicit limit

```
./sim4000 --run --config RC4000.cfg --set-disc-load-flag --steps 1000000
```

12.3. Inspect files without booting the guest

The following are separate shell invocations, each accepting one file argument. Do not combine them with interactive/run options or type them at the simulator prompt:

Host shell: independent offline inspection calls

```
./sim4000 --check-config RC4000.cfg
./sim4000 --inspect-monitor MONITOR
./sim4000 --inspect-image UNIT0
./sim4000 --list-catalog UNIT0
./sim4000 --list-main-catalog UNIT0
```

`--check-config` checks configuration syntax; it does not open all runtime media or establish that every device is implemented. Monitor/image inspection reports file properties, not successful guest operation. The catalog commands read stored records without running the monitor; they do not include an unmerged live overlay or the current guest's in-memory catalog changes. Use the guest file procedures for the running session's view.

12.4. Edit commands at the operator prompt

At the `sim4000>` prompt, `run` or `continue` gives the terminal to the RC console. Press the configured emulator escape key (Control-E by default) to suspend execution and return to the prompt. The machine, devices, console queues, and selected disk overlay remain intact.

The operator prompt uses GNU Readline. Use Left/Right and Home/End to edit, Up/Down or Control-P/Control-N to recall commands, Control-A/Control-E to move to the start/end of the line, and Control-U/Control-K to delete text. These bindings assume Readline's default Emacs editing mode; your Readline settings may change them. Control-E edits the line at this prompt, but suspends the emulator when the guest console is running. Control-D on an empty operator line exits. Nonempty consecutive duplicate commands are not added to history; the last 500 commands are kept in memory only, never written to a history file.

Tab completes commands and their aliases, `show` targets, `deposit` register names, `examine` formats, interrupt settings, media modes, and file paths in file arguments. Press Tab twice to list ambiguous matches. File paths with whitespace are not supported by the command parser and are excluded from completion. Guest commands are not completed by Readline.

Emulator output at the interactive CLI wraps at 80 columns, including long paths and completion listings. Guest terminal output is not reformatted; long edited input lines are displayed according to Readline and terminal settings. There is no separate historical TUI. Change device definitions by selecting another configuration with `-c FILE` and restarting; general device reconfiguration at the prompt is not provided. Existing media attachment commands remain available.

12.5. Simulator commands and aliases

The supported simulator commands and their explicit aliases are listed below. Parentheses, brackets, vertical bars and uppercase placeholders describe syntax; do not enter this reference block as a command file:

Simulator prompt: command and alias reference (not a script)

```
help (h, ?)
run (r)             continue (c)
stop (st)           step (s) [COUNT]   trace (t) [COUNT]
show (sh) cpu|devices|media|breakpoints|watchpoints|telnet
show throttle       set throttle VALUE
show input          discard-input (di)
show keys ADDRESS [COUNT]
examine (x) [FORMAT] ADDRESS|IC [COUNT]
deposit (d) TARGET VALUE
load-core (lc) FILE ADDRESS
save-core (sc) FILE ADDRESS COUNT
attach (at) DEVICE FILE [readonly|writable|overlay FILE|replace]
detach (det) DEVICE
rewind (rew) DEVICE
break (b) ADDRESS   nobreak (nb) ADDRESS|all
watch (w) ADDRESS   nowatch (nw) ADDRESS|all
reset (res)         start (sta)       autoload (al)
interrupts (int) on|off
attention (att)     send (se) BYTE...
send-date (sd)
quit (q)            exit
```

Addresses and counts accept C-style numbers (`123`, `0x123`). `send` accepts one or more hexadecimal 7-bit bytes, so `send 05` can deliver a literal Control-E that would otherwise be reserved by the local frontend. `send` and `send-date` append to the same local FIFO as keyboard input, behind buffered characters, and report how many bytes were queued. A full RC device queue does not reject the command; delivery proceeds during run, step, or trace. All `send` arguments are validated before any byte is queued. Explicit bytes bypass host shortcut handling and DEL/LF mappings. For example, `send 05 7f` sends literal Control-E and DEL; the RC console still interprets Escape as guest attention when appropriate. If an unprocessed lookahead byte is reported, continue or discard that input before queueing operator bytes. `discard-input` can remove these queued bytes as usual.

`reset` implements the RC4000 reset key: when leaving execution it stores the current instruction counter in word 10, disables interrupts, enters monitor mode, and preserves core and attached devices. `start` has an effect only in reset state; it reads the start address from word 14 and immediately resumes execution. `autoload` likewise requires reset state, reads the initial instruction from the paper-tape reader on device 0 into W0, and executes the bootstrap from address 0. A tape must first be mounted with `attach 0 FILE` or configured with `FILE=path` on the `DEFINE READER` line. `interrupts on|off` directly changes the stopped CPU's interrupt-enable state. Terminal attributes are restored whenever execution returns to the prompt and on handled signals.

`run`, `step`, and `trace` execute from the current instruction counter and leave reset state. Returning to the operator prompt suspends execution without resetting the guest; `show cpu` then reports `stopped=yes`. `start` and `autoload` still require reset state and use their documented hardware entry paths rather than the current instruction counter.

Interactive and bounded runs handle Control-C, termination, and hangup signals by restoring terminal settings, printing statistics, and closing media. The process exits with the usual `128 + signal` status (`130` for `SIGINT`, `143` for `SIGTERM`, and `129` for `SIGHUP`). `SIGKILL` and host process crashes cannot be cleaned up by an application handler, so a shell may still need `reset` or `stty sane` after such an uncatchable termination.

Abbreviations are explicit rather than arbitrary prefixes, so every short command has a stable meaning. `show cpu`, `show registers`, and `sh reg` are equivalent.

12.6. Modify stopped machine state

Deposits modify the live machine

The examples below demonstrate valid syntax, not a monitor-recovery sequence. They change registers and memory and can disrupt a running guest session even though execution is suspended. Use a disposable session and do not paste the block into normal work.

`deposit` changes a 24-bit core word, one of the word registers, the instruction counter, the interrupt mask or request register, or exception state while execution is suspended. The supported control targets are `IM`, `IR`, `EX21`, `EX22`, and `EX23`; exception values are range-checked. Values use the same C-style number syntax as addresses. For example:

Simulator prompt: state-changing deposit examples

```
d w0 0x123456
d ic 0x100
d im 0x9fffff
d ex22 1
d 0x200 0xabcdef
```

The instruction counter must be an even byte address. As with `examine`, an odd core address identifies the word containing that byte and is displayed in aligned form after the write.

12.7. Create a blank disk image

Disk creation is built into the simulator; no separate utility or Python is required. From the host shell, use:

Host shell: create a new disk image

```
./sim4000 --mkdisc UNIT1
```

This standalone operation does not read a configuration or start the monitor. It cannot be combined with execution or runtime options. The equivalent command at the suspended simulator prompt is `mkdisc UNIT1`; filename completion is available there. As with other simulator commands, paths at that prompt cannot contain whitespace.

The new file contains 180 zero-filled sectors of 768 bytes (138240 bytes), matching the historical starter image. It is not labelled, mounted, attached or registered in the configuration. Its initial file length is not the final logical disk capacity; writes can extend an RC433X image or overlay.

Existing files, directories and symbolic links are refused, including dangling links. There is no overwrite switch. A write failure is reported and the newly created partial file is removed when possible. Normal host file permissions and the current umask apply.

Use a new configured RC433X device and the appropriate guest disk-initialization tools before allocating guest files on it. Creating an image does not make it safe to replace an attached or mounted disk.

12.8. Inspect and change disk attachments

`show devices` lists configured device numbers, types, attachment state, and interrupt levels. `show media` reports configured media, including reader/punch files, RC749 tapes, RC4819 drives and RC433X disks. For RC433X it shows attachment state, base image, read-only, volatile, persistent-overlay or write-through mode, overlay path where applicable, and current sparse-sector count.

RC433X media can be changed while execution is suspended at the operator prompt. The three `attach` lines below are alternative modes for an already detached device, not a sequence to paste:

Simulator prompt: alternative RC433X media selections

```
detach 15
attach 15 UNIT0
attach 15 UNIT0 readonly
attach 15 UNIT0 overlay UNIT0.work.overlay
```

Suspending execution does not make disk removal safe for the guest

Detaching disc 15 from a booted monitor can remove its active backing store. Use a disposable session and account for the guest's use of the disk before changing attachments. Detach discards volatile overlay writes; reattaching the same base image does not recover them.

Disk attachment accepts only a device already declared as RC433X in `RC4000.cfg`. The default disk mode keeps writes in a volatile sparse overlay; `readonly` rejects writes, and `overlay FILE` loads or creates a

persistent sparse sidecar. These modes open the base image read-only. The additional `writable` mode instead writes directly to the selected image, like startup `--disk-write-through`; use only a disposable complete working copy, never a shipped image. Runtime paths are interpreted relative to the emulator's current directory. `detach` explicitly flushes a persistent overlay before disconnecting the device and refuses to detach if that flush fails. Volatile overlay contents are intentionally discarded on detach. RC433X operations currently complete synchronously at instruction boundaries, so there is no partially active disk command while the operator prompt is available.

13. Paper-tape files

13.1. Mount and inspect a reader tape

The configured RC2000-compatible paper-tape reader uses device 0. Its media is managed at the suspended simulator prompt, not from guest FP. The following shows media-control syntax; no guest reading occurs between these commands:

Simulator prompt: reader media controls

```
attach 0 bootstrap.pt
show media
rewind 0
detach 0
```

The reader must already be declared in the selected configuration. `attach` opens a host file, `rewind` resets the mounted reader position, and `detach` removes the media. Runtime file arguments are relative to the emulator's current directory. Mounting does not execute the tape or install guest software. Do not replace or rewind media while guest software still expects its current position.

Reader files are raw 8-bit paper-tape bytes, not text translations. Each byte contains the data holes and its parity bit; zero bytes represent blank feed and are skipped. Odd-, even-, and general-input modifiers retain the behavior of Henrik Jacobsen's original reader implementation. Autoload requires odd parity and combines the low six bits of four successive nonzero bytes into one RC4000 word. End of file produces the RC2000 end-of-paper status and leaves the reader empty. Mount, completed input, end of paper, and rewind request the configured reader interrupt.

For a tape mounted at startup, add a path to the configuration. Paths in the configuration are resolved relative to `RC4000.cfg`:

RC4000.cfg contents: mount a reader tape at startup

```
DEFINE READER DEVNO=0 INTNO=9 FILE=bootstrap.pt
```

13.2. Distinguish paper-tape autoload from disc startup

The simulator's `autoload` command requires the CPU's reset state and a reader mounted on device 0. It reads the initial word and enters the tape bootstrap; it is not the same operation as `-a`, which starts the supplied disc-based monitor session. Suspending execution alone does not put the CPU in reset state. See the simulator command reference (p. 77) before using hardware reset/start controls.

The repository does not include a historical system paper-tape image. The reader and control-panel autoload path are nevertheless covered by a generated bootstrap tape in the deterministic tests.

13.3. Create and close punch output

The RC150-compatible paper-tape punch uses device 1. It produces the same raw 8-bit tape format accepted by the reader. Its device must already be configured. These media controls create and close a file; guest output must occur while it is attached if the file is to contain punched data:

Simulator prompt: punch media controls

```
attach 1 output.pt
show media
detach 1
```

Odd- and even-parity output modes generate the parity bit from the low seven bits of each character. General output writes all eight bits unchanged. Every completed character requests interrupt 12 in the shipped configuration. Sense reports timer status while no output tape is mounted. Detach flushes and synchronizes the output before closing it and refuses to detach if that fails.

For safety, `attach 1 output.pt` creates a new file and refuses an existing path. Destructive replacement must be explicit:

Replacement destroys the previous tape contents

The `replace` form below truncates an existing output file. It is an alternative to the safe create-new form, not the next step in the ordinary workflow. Preserve wanted output first; do not use replacement to retry without checking the destination.

Simulator prompt: explicitly replace a punch output file

```
attach 1 output.pt replace
```

A `FILE=output.pt` parameter on `DEFINE PUNCH` can create and mount a new output file at startup. It uses the same no-replacement rule, and paths are resolved relative to the configuration file.

14. RC610 line printer

All supplied monitor profiles include the external process `printer`, on device 7, interrupt 10. This applies to compatibility, protected, MIPS/TS and DFC403 operation. There is no printer-free maintained monitor variant. The native emulator writes to an ordinary host file; it does not submit jobs to a host printing service.

14.1. Automatic output and file selection

The supplied configurations open output automatically in append mode:

Configuration	Output file
`RC4000.cfg`	`printer.txt`
`RC4000.protected.cfg`	`printer.protected.txt`
`RC4000.mipsts.cfg`	`printer.mipsts.txt`
`RC4000.dfc403.cfg`	`printer.dfc403.txt`

Paths are relative to the configuration file. Existing contents survive restarts; the emulator appends new prints and never truncates an output file. Each profile has its own output file. Use separate working directories for concurrent sessions; appending does not provide guest job scheduling or keep two emulators' jobs separate. The DFC403 configuration is an additional source/build-tree profile; the binary archive supplies the other three configurations.

Configuration: automatic append-mode printer

```
DEFINE RC610 DEVNO=7 INTNO=10 FILE=printer.txt APPEND=YES
```

`APPEND=NO`, or omitting `APPEND`, requires a nonexistent output file. Omitting `FILE` leaves the device installed but offline until an output file is attached. Only regular files are accepted; symbolic links, FIFOs and device files are rejected. New files have private host permissions.

To change the destination, suspend execution with Control-E and enter:

Emulator CLI: choose a new exclusive output file

```
show media
detach 7
attach 7 session-print.txt
continue
```

For an existing file use `attach 7 session-print.txt append`. Bare `attach` never overwrites an existing file. These CLI paths are relative to the emulator's working directory, unlike configuration paths. `show media` reports printer status, counts and its output path.

Wait for guest output to finish before detaching. An active hardware print cannot be detached: resume execution, let it complete, then try again. An incomplete line without a printing control is discarded and counted on close, not retroactively printed. CPU reset preserves the device and file. Normal shutdown

completes an already accepted hardware print and closes output; abnormal termination is not crash-safe spooling.

14.2. A first print from s

After date initialization, request RC attention and select `s`. The following bounded memory dump prints only the monitor's IRQ-0 vector:

Guest system process s: print a known monitor word

```
print 16 16
```

Wait for `ready`. The host printer file gains one formatted line. Repeating the command prints another line; the values depend on the monitor build. This has been checked with compatibility and protected monitors and with the DFC403 monitor. It is a memory-dump command, not file printing. Do not substitute an arbitrary user-memory range and expect a source listing.

14.3. Print an installed Algol source

The supplied disk contains `cpuregtest`, the Algol source used by `regnetest`. The tutorial example `examples/print_source.alg` opens that guest file for input and the external printer for output, copies characters up to the guest text end marker, and closes both zones. It prints the source; it does not compile or execute the numerical test itself.

Copy the example into the configured `exchange` directory as `print_source.alg`. In `s`, allocate a temporary FP:

Guest system process s: create a printing session

```
new ptest size 100000 temp disc 200 10 run
```

Guest FP, compatibility profile: compile and print

```
psrc=fromhost "print_source.alg"
ptext=algol psrc connect.no
ptext
```

For the protected profile, start with `RC4000.protected.cfg` and its matching monitor/disk pair. Use the same example and `s` allocation, but import through the protected profile's transfer program:

Guest FP, protected profile: compile and print

```
psrc=rcin "print_source.alg"
ptext=algol psrc connect.no
ptext
```

Both checked profiles print 630 lines/printing operations and 17,309 bytes, beginning with the MPROFILE header. The output is visible before closing the emulator. The tiny program uses `open(source,4,...)` for the disk area and `open(printer,0,...)` for the physical printer; selecting a disk output called `printer` is not the same operation. The FP recipes `o printer` and `printer=copy cpuregtest` are not accepted substitutes: exploratory runs did not drive the hardware printer.

Both runs match an independent whole-source guest export: `tohost` in compatibility mode, `rcout` in protected mode. The protected check keeps `PROTECTION MODE=RC4000` enabled and observes keyed task instructions during execution of the printing program, not merely during boot. It uses the existing protected libraries without changes to the example or images. Enter `finis` after printing and wait for `pause ptest finis fp`. Normal return to `s` and a subsequent `list` are also checked.

14.4. Text mapping, visibility and limits

The host file is a print-operation stream, not a rendered paper image. A printing control emits the buffered line followed by its original control:

Guest control	Host byte	Meaning
NL (10)	LF	Begin the next line.
VT (11)	VT	Retain vertical-tab motion.
FF (12)	FF	Retain the form/page separator.
CR (13)	CR	Return on the same line for overprinting.

No LF is inserted after CR. VT/FF remain literal controls; no physical format tape is emulated. Text viewers may therefore display overprints and page controls differently. The buffer has 132 columns; excess graphics are lost and counted. Unsupported standard-character-set codes and optional HT/SO/SI/CAN/ESC functions are ignored. National RC character positions remain raw seven-bit bytes, not UTF-8 Danish letters.

Every completed print calls `fflush` before its completion interrupt, including CR, VT and FF. Output does not wait for a full host buffer or emulator exit. A guest program can still buffer text before sending it to the printer. There is no per-line `fsync` durability guarantee. Host write/flush errors are latched as virtual offline/paper-out status, not silently ignored or replayed; choose a healthy new output after resolving the host error.

The timing model is deterministic, not mechanically accurate. A print completes on the next positive device advance. The supplied MIPS/TS profile supports PRIMO transports to the real external process `printer`; see Print through PRIMO (p. 64). Repeated jobs, signoff and restart append are checked. Recovery of a transport submitted while output is detached is described in Recover a job submitted with output detached (p. 65). General host-write failure recovery and arbitrary queue capacity remain separate tasks. The MIPS/TS chapter records the bounded two-client check and its second blocking-wait restriction.

15. Optional V2 terminal protocol

15.1. Keep classic operation as the normal profile

The existing terminal behavior remains the default (`PROTOCOL=CLASSIC`). An experimental protocol based on Jacobsen's V2 terminal code can be enabled independently on a `CONSOLE` or `RC851` declaration in a separate test configuration, for example:

Separate test configuration: an additional V2 endpoint

```
DEFINE CONSOLE DEVNO=7 INTNO=14,13 PROTOCOL=V2
TELNET DEVNO=7 ADDRESS=127.0.0.1 PORT=4007 ATTENTION=CTRL-A
```

This snippet belongs to a separate test configuration. It creates another device/endpoint; it is not a runtime command or a replacement for the primary console. Configuring the endpoint does not install a V2 monitor or a matching guest driver.

V2 is a guest-device protocol, not a Telnet upgrade

Choose free device and interrupt numbers; these are free in the shipped default configuration when its other optional terminal lines are inactive. `show devices` and `--check-config` identify the selected V2 protocol. Do not change the primary terminal of the supplied monitor to V2 just to obtain more terminal features: its existing driver expects the classic protocol. V2 requires guest software/monitor drivers that understand it. For example, a classic read returns CR as 13; V2 table 1 returns terminating LF with status bit 20 (`0x10000a`). The supplied typewriter driver performs direct character comparisons and its own editing/conversion, so these are not interchangeable results even if ordinary printing still works.

15.2. Understand input tables and frontend policies

In V2 mode the guest can set/query terminal type and prompt, suppress input echo for blind reads, suppress only backspace echo, and write the configured prompt character. There are three input tables:

Type	Intended use	Control-character handling
0	RC4000 console	CR becomes terminating LF; backspace, cancel and attention are flagged.
1	DTU data entry (initial selection)	Most control keys terminate input without echo; backspace/cancel retain special handling.
2	Screen editor	Most control keys terminate input without echo; DEL is returned as 127.

The initial prompt is BEL (7), as in the V2 source. Type selection is a guest attribute, not ANSI/VT100 terminal detection. Input echo occurs once when the guest senses a completed read. Existing keyboard/date shortcuts, DEL mapping, Telnet framing and ANSI filtering remain frontend policies: they apply before the device's input table. Use `INPUT DEL=NONE` if the V2 table must receive a literal DEL, and avoid reserving a Telnet attention key needed by the guest.

15.3. Observe the experimental compatibility boundary

This block does not implement interrupt expanders, guest-controlled Telnet disconnect, DOS serial/multiport hardware or the V2 host-file protocol. The RC851 output translation remains the existing ANSI/VT100 subset; enabling V2 does not certify full-screen editor compatibility. Tests cover synthetic guest I/O and mixed classic/V2 configurations, not booting the V2 monitor.

For ordinary monitor sessions, use the classic-profile instructions in the Telnet chapter (p. 97). The type-2 table's screen-editor intent does not establish that the installed historical editor is compatible; retain the manual's inventory-only boundary for unverified screen-editor operation.

16. Magnetic-tape files (RC749)

16.1. Enable and mount an RC749 device

RC749 is an optional read-only magnetic-tape device based on the additional protocol in Henrik Jacobsen's `V_1.2_DB` emulator. It is distinct from the paper-tape reader. Enable it with a free device number and interrupt level; the following are free in the default configuration:

RC4000.cfg contents: declare an operator-controlled RC749 device

```
DEFINE RC749 DEVNO=16 INTNO=11
```

Restart with that device declaration before using it. At the suspended simulator prompt (not guest FP), these are media-control examples; they do not read or install the software themselves:

Simulator prompt: RC749 media controls

```
attach 16 software.tap
show media
rewind 16
detach 16
```

`show media` reports the file, logical record position, record count, status and transfer counters. Position zero is the beginning of tape. Tape marks count as records; erase-gap markers do not. Detach before replacing a tape. RC749 does not accept writable or overlay modes. The host file is always opened read-only, and guest write/erase commands report intervention rather than pretending to succeed.

16.2. Recognize the supported tape format

For a startup tape, add `FILE=tapes/software.tap`. Configuration paths are relative to the configuration file; operator `attach` paths are relative to the current working directory. The accepted format is tagged TAP: 32-bit little-endian lengths surrounding each data record, a padding byte after odd-length payloads, zero-length tape marks, erase-gap and end-of-medium markers, and record-error flags. It is not a raw byte stream or a guarantee of support for every extended TAP variant. Mount rejects truncated or mismatched records. Record transfers are limited to 7,680 bytes; larger records are skipped with block-length status.

A mounted tape is not an installed guest program

The extension `.tap` alone does not establish the record format or guest compatibility. Keep archived originals unchanged, inspect the format before mounting, and do not rewrite a magnetic-tape container as raw paper-tape bytes. Rewind changes tape position, not the CPU or guest process state.

Guest software can read records, sense status and byte counts, space forward or backward by record/file, rewind and unload. This does not automatically install software or change the bundled monitor's device definitions. Using an archived tape requires compatible guest software and monitor definitions; no historical software tape is bundled or verified yet.

16.3. Opt in to guest-requested tape mounting

The DB emulator also repurposes output-even as a guest-requested mount. This extension is disabled unless an explicit `ROOT` is configured:

RC4000.cfg contents: opt in to guest-requested mounting

```
DEFINE RC749 DEVNO=16 INTNO=11 ROOT=tapes
```

This is an alternative device declaration, not a second device with the same number. The root directory must already exist; a relative `ROOT` is resolved against the selected configuration file. It authorizes compatible guest software to request tapes beneath that directory without an operator `attach` command, but provides no guest installer or automatic mount dialog.

Guest filenames must be relative Unix paths beneath that directory. Absolute paths, `./..` components, symlinks and non-regular files are rejected. A failed replacement leaves the previously mounted tape intact and reports an error. Omit `ROOT` if only operator-controlled mounts are wanted. No host environment variables or historical drive-letter paths are used.

17. Debugging and core files

All commands in this chapter are simulator commands typed at the suspended `sim4000>` prompt, not guest FP commands. They operate on the current machine session. Addresses are RC byte addresses; one 24-bit core word occupies two RC bytes. Numeric arguments accept the C-style notation used in the operator reference (p. 77).

17.1. Stop before execution or after a core write

`break` installs an execution breakpoint at an even byte address. Execution stops before the instruction at that address, so registers and memory still describe its input state. `continue` executes past a breakpoint once; a later visit to the same address stops again. Breakpoints can be inspected and removed with:

Simulator prompt: inspect and remove execution breakpoints

```
show breakpoints
nobreak 0x100
nobreak all
```

`watch` installs a core-write breakpoint at an even byte address. It observes CPU stores and device transfers through the common machine-memory boundary. Execution stops after the instruction or device operation has completed and reports the previous and resulting 24-bit word. Register writes and operator `deposit` commands do not trigger core-write breakpoints. Use `show watchpoints`, `nowatch ADDRESS`, or `nowatch all` to inspect and remove them.

17.2. Inspect core without modifying it

`examine` defaults to hexadecimal words and accepts four explicit formats:

Simulator prompt: alternative core-display formats

```
x /hex 0x200 8
x /char 0x200 8
x /binary 0x200 4
x /instruction ic 10
```

These four display calls are alternatives for different views of core; they neither run instructions nor change memory. Counts select words, not host file bytes.

Character view shows the three high-to-low bytes of each 24-bit word and replaces non-printable bytes with periods. Instruction view uses SLANG-style mnemonics and addressing notation. `IC` follows the current instruction counter, and the word at the current instruction counter is marked with `>` in every view.

`show cpu` also reports the protection profile and PR. `show keys ADDRESS [COUNT]` reads the word keys without changing them; COUNT defaults to one and is limited to 256. Odd addresses select the containing

word. `PR-protected` describes the selected PR bit, not whether checks are enabled or the current monitor mode permits access. The supplied profile leaves protection off. For example:

Simulator prompt: read protection state without executing

```
show cpu
show keys 0 4
```

17.3. Exchange packed core data with host

`load-core` and `save-core` exchange contiguous core words with host files. The file format is headerless: each RC4000 word is stored as exactly three bytes, most-significant byte first. This is the same 24-bit packing used by the supplied `MONITOR` file. Addresses must be even byte addresses; counts are word counts. Loading rejects empty, truncated, and out-of-range files before changing core. Saving uses a temporary file in the destination directory and atomically replaces the named file only after all words have been written and flushed. File paths are relative to the emulator's current directory, not the `HOSTFILES` exchange root. For example:

Core files are not session snapshots

`load-core` overwrites the target core words; it can corrupt a live monitor or guest program even while execution is suspended. `save-core` can replace an existing host destination, unlike guest `tohost` exports. Choose a disposable session and an unused destination for the example below. The file contains only the selected core words, not registers, devices, queues or disk-overlay state; loading it does not restore a complete session.

Simulator prompt: save, load and inspect packed core words

```
save-core bootstrap.bin 0 16
load-core bootstrap.bin 0x200
x /hex 0x200 16
```

The example saves 16 words (48 host bytes) and loads them at RC byte address `0x200`. It illustrates copying data, not setting a valid program entry point or preparing a runnable monitor.

Core-data operations are available only at the suspended operator prompt and do not change the instruction counter or trigger write watchpoints.

17.4. Execute a bounded diagnostic step

`trace` is a bounded diagnostic form of `step`. It prints the executed instruction's address and raw word, decoded opcode and addressing fields, effective address, and resulting instruction counter. A count defaults to one, for example `t 10`. Address-modification prefixes consumed by one interpreter call are reported on the resulting instruction's trace line.

Tracing executes instructions

Unlike `examine /instruction`, `trace` changes the machine state and can perform device I/O or guest disk writes. A step count bounds interpreter calls, not an entire guest operation. Use disk-copy or overlay precautions appropriate to the running configuration; a short trace is not a read-only inspection.

18. Execution throttle

18.1. Choose an interactive pacing mode

The shipped configurations use a 50% CPU budget for interactive execution. At `sim4000>`, inspect or change it with the following alternatives. Enter one `set` command for the mode you want; these are simulator commands, not guest FP input:

Simulator prompt: inspect throttle and choose one pacing mode

```
show throttle
set throttle 25%
set throttle 250000ips
set throttle original
set throttle off
```

- `1%` through `100%` budget that fraction of one host CPU, measured using process CPU time against monotonic elapsed time. This is not a percentage of historical RC4000 speed. The limit is approximate over short intervals; scheduling and I/O can make actual usage lower. `100%` does not insert waits.
- `1ips` through `1000000000ips` limit actual RC instructions per second, including each `am` prefix, not interpreter calls. A slow host may not reach the requested rate.
- `original` is a rough average-speed preset equivalent to `250000ips`, based on the typical 4 microseconds per instruction described in the historical *RC 4000 Software: Multiprogramming System* (general description and section 9.4). It is not an opcode-by-opcode timing model. The June 1969 reference manual, appendix A.3, gives differing instruction and addressing times; floating-point-heavy workloads in particular may differ substantially from real hardware. Disk seek/rotation delays are not introduced by this preset.
- `off` removes deliberate execution waits.

18.2. Keep a startup setting in the configuration

To select a startup value, use exactly one active directive in the config:

RC4000.cfg contents: one active throttle directive

```
THROTTLE VALUE=50%
; THROTTLE VALUE=ORIGINAL
; THROTTLE VALUE=250000IPS
; THROTTLE VALUE=OFF
```

Configuration values are case-insensitive. Old configurations with no directive remain unthrottled. Runtime changes last only for the current session and do not rewrite the configuration. Tab completion supports the setting and common values.

18.3. Understand what pacing does not control

Pacing applies to `run`, `continue`, and `-a` autostart. Step and trace are unpaced debugging operations; the bounded `--run` runner deliberately ignores throttle settings to preserve fast, deterministic instruction-count tests. The real-time interactive device clock still follows host elapsed time. Throttle waits service console input, Telnet and the device clock in short intervals, so the configured emulator escape key (Control-E by default) and handled signals can interrupt even a `1ips` wait. Returning from the operator prompt starts a fresh pacing budget; no unused time from command editing is accumulated as execution credit.

Approximate historical speed is not cycle accuracy

Choose a percentage budget to reduce host CPU use, or `original` for the average-rate approximation. These are different goals: a fixed instruction rate does not promise a particular host CPU percentage. Neither mode reproduces opcode-specific timing, physical disk delays or historical peripheral throughput, and `--run` does not use either limit.

---

19. Keyboard and date settings

19.1. Set local shortcuts and initialize the date

The following settings apply to the local interactive guest console while the emulator is running. Change them in your configuration before startup; they do not redefine GNU Readline's operator-prompt bindings. The shipped active directives are:

RC4000.cfg contents: local input and date settings

```
OPERATOR ESCAPE=CTRL-E
OPERATOR DATE=ALT-D
INPUT DEL=CTRL-H
DATE YEAROFFSET=-40
```

The escape setting accepts a control-letter key or `NONE`. With escape disabled, that key is delivered as guest input rather than opening the operator prompt. Leave an accessible escape key configured for normal interactive work. The date shortcut is an Alt-letter key or `NONE`; neither setting requires the historical TUI.

At the monitor's `initialize date:` prompt, the configured Alt-D shortcut inserts the current local host date in the historical `YY MM DD HH MM SS` format. `OPERATOR DATE=ALT-D` can select another Alt-letter or `NONE`. `DATE YEAROFFSET=-40` translates 2026 to 1986; only translated years 1982 through 1999 are accepted. The same insertion is available portably by using the configured emulator escape key (Control-E by default), `send-date`, and `continue`. The date may also be typed manually. Choose the year offset for the historical year you intend to use; `-40` is a fixed translation, not a setting that keeps the guest year constant as the host year changes. This shortcut inserts console input; it does not set the monitor date independently of the guest's initialization dialog.

19.2. Map Backspace and decode terminal input

Modern terminals normally send DEL (`0x7f`) for the Backspace key, while the RC software expects Control-H (`0x08`). `INPUT DEL=CTRL-H` performs that mapping; `INPUT DEL=NONE` passes DEL through unchanged.

You can paste a complete guest command line, including lines longer than the emulator's small console queue. Input waits for queue space without disabling the execution throttle. In FP, enter or paste one command at a time and wait for it to finish; FP has no prompt. This is not a general multi-command script interface: a program may change command context or ask for additional input. The configured operator escape (Control-E by default) is recognized even when the guest input queue is full. It suspends execution without discarding guest input already read by the emulator; `continue` delivers the buffered characters in their original order. The escape itself is not sent to the guest. With `OPERATOR ESCAPE=NONE`, that control byte remains guest input. Avoid pasting input into a program that does not request it: suspension does not cancel it. Control-C remains a host signal that exits the emulator rather than suspending it; volatile disk writes are lost on exit as usual.

The local interactive console accepts ASCII, not UTF-8. Non-ASCII bytes are rejected rather than silently changed into unrelated guest characters; the emulator reports the rejected byte count when execution is suspended. Common ANSI cursor/function sequences, terminal control strings, and bracketed-paste start/end markers are consumed without invoking guest attention. ANSI Delete (`ESC [ 3 ~`) follows `INPUT DEL`. Paste contents still follow ordinary input rules, including the configured operator escape and date shortcut. This does not add cursor editing to the guest monitor or enable bracketed-paste mode in the terminal; it safely handles framing if the terminal supplies it. Bare Escape still requests historical guest attention: the frontend waits up to 30 ms for its next byte to distinguish an Escape-prefixed sequence or Alt shortcut. Unusually delayed prefixes can therefore be interpreted as bare Escape; use the operator `attention` command for an unambiguous request. Sequence bodies may span input-service calls. `send` bypasses terminal decoding. Data-only bounded `--run` input retains its existing byte-oriented behavior.

19.3. Inspect or discard pasted input

After suspending execution, `show input` reports buffered input for the local primary console: `host-buffer` is the host-side FIFO, `pending-byte` is an unprocessed lookahead byte, and `guest-queue` is the RC console device queue. `completed-read=yes` means a character has already completed a device read but has not yet been sensed by the guest. `eof` describes the host input stream. The output is limited to 80 columns like other simulator diagnostics. `terminal-sequence` indicates an unfinished host sequence, and `rejected-non-ascii` is the cumulative rejected byte count for this session.

To discard an accidental paste, press the configured escape key first (Control-E in the example), then enter the simulator commands below one at a time. The angle-bracket line denotes a keypress, not literal command text:

Local escape key, then simulator prompt: inspect and discard input

```
<Control-E>
show input
discard-input
show input
continue
```

Discarding queued input does not undo guest input

`discard-input` (alias `di`) reports the discarded byte counts and clears only the local host FIFO, lookahead byte, unfinished terminal sequence, and local RC device queue. It does not send attention, reset the machine, touch other terminals, or change disk files. This discard is irreversible. Repeating it with empty queues discards zero bytes. Characters not yet read from the host terminal and Readline's edited command line are not affected.

A completed device read, its interrupt, and characters already received by the guest are preserved. Thus this command does not undo a partial FP command or cancel a running guest program. Use the guest's erase/attention mechanisms as appropriate to its current context before entering a replacement command.

Interactive execution advances the 39 Hz device clock from monotonic host time. This keeps console input timeouts usable at human typing speeds. The bounded `--run` mode retains instruction-count-based deterministic timing for regression tests.

19.4. Keep operator input separate from bounded-run data

For automated CPU and device regression tests, `--trace FILE` records every bounded-run interpreter call and every core-store write in a stable, tab-separated format. The output file must not already exist:

Host shell: bounded run with a new trace file

```
./sim4000 --run --steps 1000 --trace startup.trace
```

The trace is an ASCII, tab-separated diagnostic file with LF line endings. Its first line identifies the format and version. Records beginning with `S` contain CPU state after an interpreter call; `W` records describe core-store writes. Traces can grow quickly. For a meaningful comparison, keep the monitor, disk images, configuration, input and step limit identical, and avoid asynchronous terminal or Telnet input. A trace is not a disk backup or a resumable saved machine state.

Interactive mode intentionally requires a terminal on standard input. Piped input remains console data for the bounded runner and is never interpreted as operator commands.

The local date shortcut and `send-date` workflow target the local primary console. Do not assume these are remote Telnet shortcuts; see Telnet attention and command contexts (p. 97) for remote operation.

20. Telnet consoles

For the preconfigured MIPS/TS service on port 4004, follow Connect and log in to MIPS/TS (p. 54); do not create another listener with the same device or port. This chapter describes the general console mechanism, not the SOS account setup.

20.1. Enable a listener and connect a client

An additional configured `CONSOLE` can be exposed as a data-only ANSI/VT100 terminal over Telnet/NVT. The service is disabled by default. Enable the commented loopback example in `RC4000.cfg`:

RC4000.cfg contents: enable the existing device-4 endpoint

```
TELNET DEVNO=4 ADDRESS=127.0.0.1 PORT=4004 ATTENTION=CTRL-A
```

The endpoint must appear after its corresponding `DEFINE CONSOLE` line. Start the emulator in one host terminal, then connect an ANSI-capable Telnet client from a second host terminal. These are separate shell invocations:

First host terminal: start the emulator

```
./sim4000 -a
```

Second host terminal: connect to the remote guest console

```
telnet 127.0.0.1 4004
```

Device 2 remains the local console and device 4 is isolated on its Telnet port. The listener is nonblocking, accepts one active client, performs Telnet option and NVT Return handling, and buffers console input. A second simultaneous client is rejected. DEL uses the configured console input mapping. Telnet does not expose emulator commands or the Control-E operator prompt. Listeners continue to accept, reject, disconnect, and reconnect clients while the CPU is stopped at `sim4000>`. Use `show telnet` there to inspect each endpoint's bind address, connection state, attention mode and count, buffer occupancy, and connection/drop counters.

Each endpoint exposes its configured guest device, not a shared operator console. Keep the example bound to loopback for local use: the service provides neither authentication nor transport encryption. An endpoint bound to another address must be protected by the host's network-access controls; connecting does not require a guest login provided by this emulator.

Telnet buffers are bounded

Input and output can lose bytes when their buffers fill; inspect the endpoint's drop counters with `show telnet` at the local simulator prompt. Send one guest command at a time and wait for its complete reply, especially while execution is suspended or the client is slow to read. Reconnecting does not reconstruct output that was dropped or discarded on disconnect. The local `discard-input` command does not clear a remote terminal's queued input.

20.2. Request guest attention, not emulator suspension

Control-A is the default remote RC operator key. It is intercepted by the endpoint, is not delivered as RC console data, and avoids entering the Telnet client's command mode merely to issue `send brk`. Telnet `BRK` remains supported in every mode. The alternatives are configured per endpoint:

Endpoint option alternatives: choose one ATTENTION value

```
ATTENTION=CTRL-A   request attention when Control-A is typed (default)
ATTENTION=CONNECT  request attention whenever a client connects
ATTENTION=NONE     disable the in-band key; Telnet BRK still works
```

These `ATTENTION=` values are alternatives inside a `TELNET` declaration, not complete configuration lines or commands to type into the connection. A remote Control-E does not open the local `sim4000>` prompt; remote input belongs to the guest terminal. Use the local emulator escape key for simulator control.

20.3. Select a process on a second terminal

Attention displays `att`, after which the terminal selects an available RC process by name. A process can be bound to only one terminal. The transcript labels `local:` and `remote:` identify where to type; they are not command text. The local starting context is the supervisor `s`; choose unused names for the example processes. For example, this verified two-terminal workflow first moves the local console away from `s` and then uses `s` remotely:

Two-terminal transcript: move the supervisor to the remote console

```
local:  new telproc size 100000 run
remote: <Control-A>
remote: s
remote: new remote size 100000 run
remote: geo
```

The remote terminal reaches `dansk/English ?`; enter `e` to reach `ARE YOU READY?!`. This is a guest program question, not the Telnet client's command prompt. Only one client can use an endpoint at a time. Use `show telnet` at the local simulator prompt to inspect rejection and connection counters. Disconnecting and reconnecting the client does not itself remove its guest process.

Keep local and remote FP processes independent

For a separate two-terminal process-control exercise with MONITOR/RC433X, start a fresh session. With the local terminal selected to `s` and a connected device-4 Telnet console, the following sequence before starting `geo` is:

Two-terminal transcript: independent FP process lifecycle

```
local:  new local size 100000 run
remote: <Control-A>
remote: s
remote: new remote size 100000 run
local:  montest internal name.local
remote: montest internal name.remote
remote: <Control-A>
remote: s
```

```
remote: proc remote remove
remote: <Control-A>
remote: s
remote: list
remote: <Control-A>
remote: s
remote: new remote size 100000 run
```

The prefixes and `<Control-A>` notation identify the terminal or key; they are not literal command text. Wait for each operation's complete output, especially `ready` after removal and `list`, before requesting attention again. The completed list still contains `local` and no longer contains `remote`.

Keep removal separate from reconnection

Only remove a process whose temporary work you are prepared to lose. This example deliberately removes and recreates `remote`; it is not a harmless reconnect procedure. A Telnet disconnect does not itself perform `proc remote remove`. Keep terminal reconnection separate from guest process administration. Direct removal is not a file-cleanup guarantee: clear unwanted temporary files inside FP first if you need predictable catalog cleanup.

The two montest calls identify the separate `local` and `remote` processes. After removal and recreation, the local process remains available; the new remote process is ready for FP commands. Process addresses can change. For a demonstration, start `geo` separately in each idle FP only after finishing the process-control exercise.

This is operation on two separate guest terminals, not two clients sharing one Telnet endpoint or two FPs competing for one console's pending reads. It is checked with the default monitor/device profile; the analogous live Telnet lifecycle has not been checked with STDMON/DFC403. It does not validate arbitrary process trees, storage isolation or protection.

20.4. Use the RC851 screen translation

`RC851` devices use the same endpoint syntax and translate their historical screen protocol to ANSI/VT100. The implemented output operations include cursor addressing, cursor forward/up, Home, four-column Tab, clear screen, clear line, erase-to-end-of-line, erase-to-end-of-screen, Return, Line Feed, Backspace, and Bell. For example:

RC4000.cfg contents: declare an RC851 screen endpoint

```
DEFINE RC851 DEVNO=5 INTNO=16,15
TELNET DEVNO=5 ADDRESS=127.0.0.1 PORT=4005 ATTENTION=CTRL-A
```

Common ANSI cursor and function-key sequences on input are consumed rather than leaking their Escape byte into the RC operator-key path. Printable input, Return, Backspace, and Delete mapping remain available. Additional key mappings can be added when an RC workload demonstrates a need for them.

Index

Algol, 16
Attention, 97

Binary export
  SLANG, 34
Binary transfer, 50
Breakpoints, 75
  execution timing, 90
Building the emulator, 2

cata, 24
Catalog, 18
  free slots, 24
  offline inspection, 24
  stored records, 74
catsort, 22
catstat, 26
claim, 11
clear, 17
cmp
  guest text comparison, 32
Command contexts, 3, 7
Command files, 35
Command-line options, 75
  bounded execution, 75
  offline inspection, 76
  path overrides, 75
  startup modes, 75
Completion, 75
Configuration, 70
  compatibility and protection, 70
  CPU and core size, 70
  relative paths, 72
  syntax checking, 76
continue, 75
copy, 15
Core files, 90
CPU budget, 92
cpuregtest, 44

Date
  initialization, 3
  setting, 12
  shortcut, 94
  year offset, 94
Debugging, 90
deposit, 78
DFC403, 72
Diagnostics
  numerical workload, 44
Disk attachments
  operator commands, 79
Disk persistence, 70
  mode choice, 73
Disks
  additional RC433X disk, 71
  creating a blank image, 79

Editor, 30
  deletion, 32
  finishing, 32
  insertion, 32
  line movement, 32
Error recovery, 41
  assembly error, 41
  IX-disabled catstat, 26
  missing program, 40
examine
  display formats, 90

File retention, 7, 18
Files
  copying and renaming, 15
  import and export, 15
finis, 12
FP, 10
  command-file input, 36
  conditional commands, 38
  interrupt running program, 36
  nested command files, 36
  output redirection, 35
  result status, 38
fromhost, 50
  text import, 51

Host file exchange, 50
  conversion modes, 51
  file permissions, 52
  non-overwrite protection, 52
  path restrictions, 50
  text comparison, 34
HOSTFILES
  exchange directory, 50

if (FP command), 38
Input queues, 95
IX, 26

Keyboard, 94
  Backspace mapping, 94
  local shortcuts, 94
  terminal sequences, 94

kitlabel, 71
kiton, 71

load-core, 91
lookup, 15

Magnetic tape, 88
MIPS/TS, 53
  backup, 61
  batch jobs, 57
  failure diagnosis, 67
  normal shutdown, 61
  partial input after reconnection, 62
  persistence, 59
  resource claims, 56
  retained-file exercise, 59
  Telnet reconnection, 62
  troubleshooting, 66
mkdisc, 79
montest, 45
  command and record input, 45
  disk-chain header, 46
  external device descriptors, 46
  help, 45
  internal process descriptors, 46

new, 10

Output redirection
  restoring the console, 36
Overlays, 70

Paper tape, 81
  autoload, 81
  explicit replacement, 82
  punch output, 82
  reader attachment, 81
PRIMO, 53
  printing, 64
Printer, 83
Processes, 10
  shared terminal, 13
Protection
  monitor profile, 70
Punch, 81

RC4819, 72
RC610, 83
RC749, 88
  guest-requested mounting, 89
  operator-controlled mounting, 88
  record position, 88
  TAP format, 88
RC851
  ANSI translation, 99
Reader, 81
Readline, 75
regnetest, 44
remove, 12
rename, 15
run, 75
Runtime files, 5

save-core, 91
scope, 19
search, 21
show
  guest disk space, 24
Shutdown, 7
Simulator commands, 75
  aliases, 77
SLANG, 34
  conditional export, 39
  execute sample, 36
  listing, 35
Software inventory, 28
SOS, 53
  batch jobs, 58
  password forms, 58
Startup, 2
  recovery, 4

Telnet, 97
  attention key, 98
  connection state, 97
  listener configuration, 97
  process lifecycle, 98
  process selection, 98
TEM, 53
Terminal protocol
  input tables, 86
  monitor compatibility, 86
  V2, 86
Terminals, 97
Text transfer, 50
Throttle, 92
  configuration, 92
  runtime settings, 92
  timing limits, 93
tohost, 50
  text export, 51
Tracing, 90
  instruction execution, 91

Watchpoints
  core writes, 90
Working directory, 5
Write-through, 70
  working copy, 73